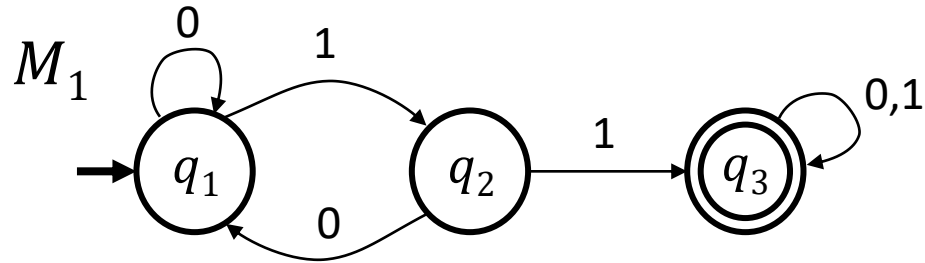


Finite Automata

Acknowledgements

- The content of the following slides is based on
 - The textbook: M. Sipser, Introduction to the theory of computation, Third edition, Cengage Learning, 2013.
 - The lecture slides by Prof. M. Sipser:
<https://math.mit.edu/~sipser/18404/Lectures%20Fall%202020/index.html>

Finite Automata



States: $q_1 q_2 q_3$

Transitions: $\xrightarrow{1}$

Start state: $\rightarrow \bigcirc$

Accept states: $\bigcirc\bigcirc$

Input: finite string

Output: Accept or Reject

Computation process: Begin at start state, read input symbols, follow corresponding transitions, Accept if end with accept state, Reject if not.

Examples: 01101 $\rightarrow$ Accept

00101 $\rightarrow$ Reject

M_1 accepts exactly those strings in A where
 $A = \{w \mid w \text{ contains substring } 11\}.$

Say that A is the language of M_1 and that M_1 recognizes A and that $A = L(M_1).$

Finite Automata – Formal Definition

Defn: A finite automaton (aka deterministic finite automaton: DFA) M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

Q finite set of states

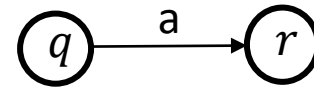
Σ finite set of alphabet symbols

δ transition function $\delta: Q \times \Sigma \rightarrow Q$

q_0 start state

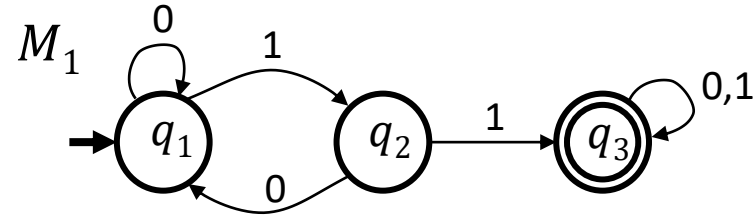
F set of accept states

$\delta(q, a) = r$ means



Finite Automata – Formal Definition

Example:



$$M_1 = (Q, \Sigma, \delta, q_1, F)$$

$$Q = \{q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_3\}$$

$\delta =$	0	1
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_3	q_3

Finite Automata – Computation

Strings and languages

- A string is a finite sequence of symbols in Σ
- A language is a set of strings (finite or infinite)
- The empty string ϵ is the string of length 0
- The empty language $\emptyset$ is the set with no strings

- $L(M) = \{w \mid M \text{ accepts } w\}$
- $L(M)$ is the language of M
- M recognizes $L(M)$

Def: A language is regular if some finite automaton recognizes it.

Def: M accepts string $w = w_1w_2 \dots w_n$ each $w_i \in \Sigma$ if there is a sequence of states $r_0, r_1, r_2, \dots, r_n \in Q$ where:

- $r_0 = q_0$
- $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$
- $r_n \in F$

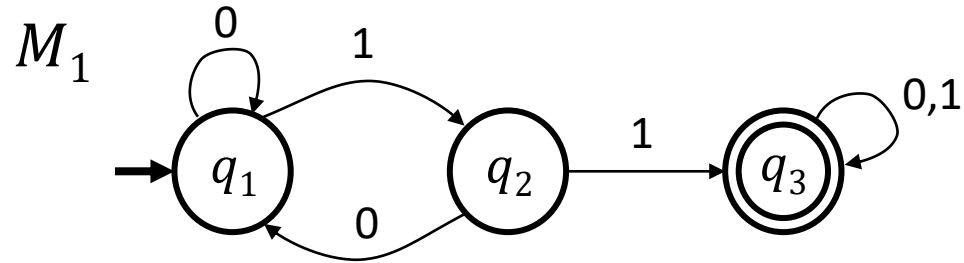
Finite Automata – Computation

- $L(M) = \{w \mid M \text{ accepts } w\}$
- $L(M)$ is the language of M
- M recognizes $L(M)$

Def: A language is regular if some finite automaton recognizes it.

Def: Two automata M and M' are said to be equivalent if and only if
$$L(M) = L(M')$$

Regular Languages – Examples



$$L(M_1) = \{w \mid w \text{ contains substring } 11\} = A$$

Therefore A is regular

More examples:

Let $B = \{w \mid w \text{ has an even number of 1s}\}$
 B is regular.

Let $C = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$
 C is not regular (we will later prove).

Goal: Understand the regular languages

Regular Operations

Regular operations. Let A, B be languages:

- Union: $A \cup B = \{w \mid w \in A \text{ or } w \in B\}$
- Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\} = AB$
- Star: $A^* = \{x_1 \dots x_k \mid \text{each } x_i \in A \text{ for } k \geq 0\}$
Note: $\varepsilon \in A^*$ always

Example. Let $A = \{\text{good, bad}\}$ and $B = \{\text{boy, girl}\}$.

- $A \cup B = \{\text{good, bad, boy, girl}\}$
- $A \circ B = AB = \{\text{goodboy, goodgirl, badboy, badgirl}\}$
- $A^* = \{\varepsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, ...}\}$

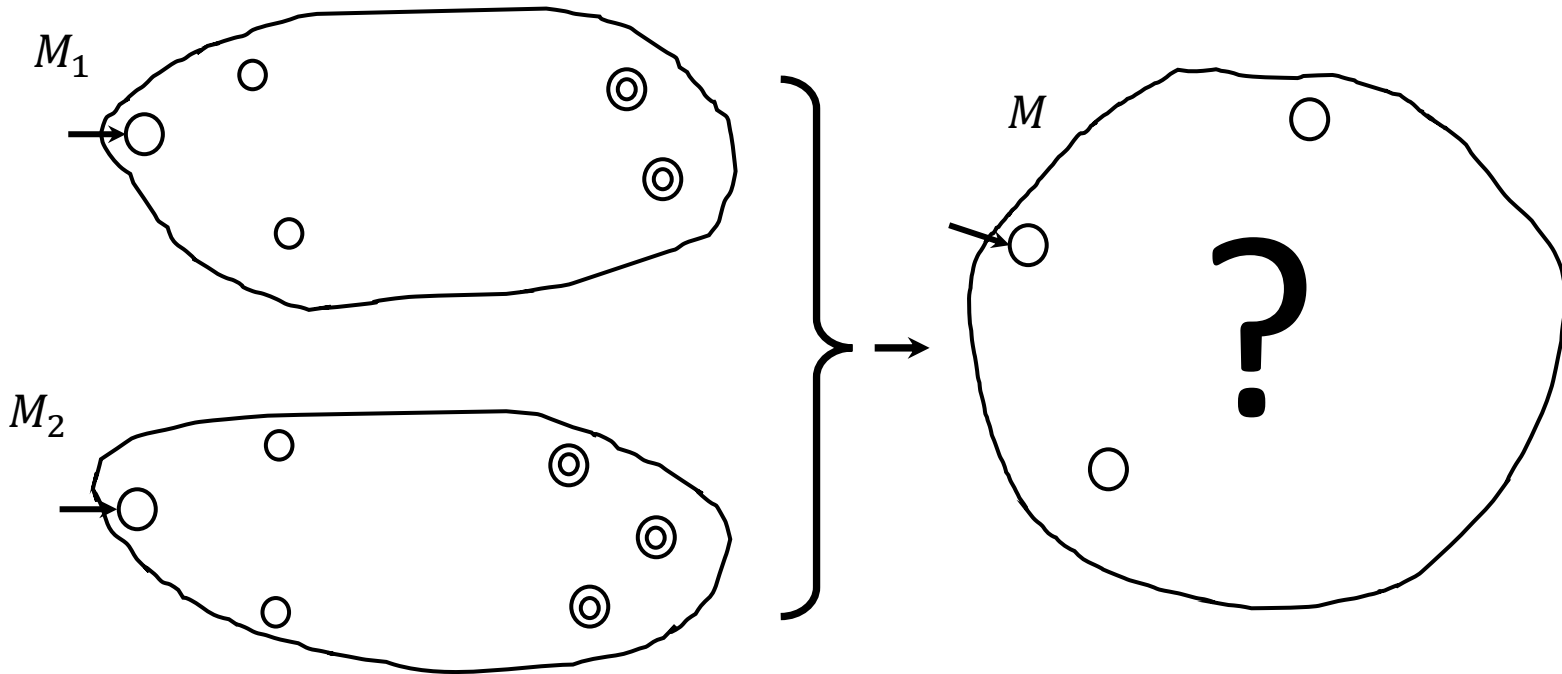
Closure Properties for Regular Languages

Theorem: If A_1, A_2 are regular languages, so is $A_1 \cup A_2$ (closure under $\cup$)

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1

$$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$$

Construct $M = (Q, \Sigma, \delta, q_0, F)$



Components of M :

$$Q = Q_1 \times Q_2 \\ = \{(q_1, q_2) | q_1 \in Q_1 \text{ and } q_2 \in Q_2\}$$

$$q_0 = (q_1, q_2)$$

$$\delta((q, r), a) = (\delta_1(q, a), \delta_2(r, a))$$

$$F = F_1 \times F_2 \text{ NO! [gives intersection]}$$

$$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

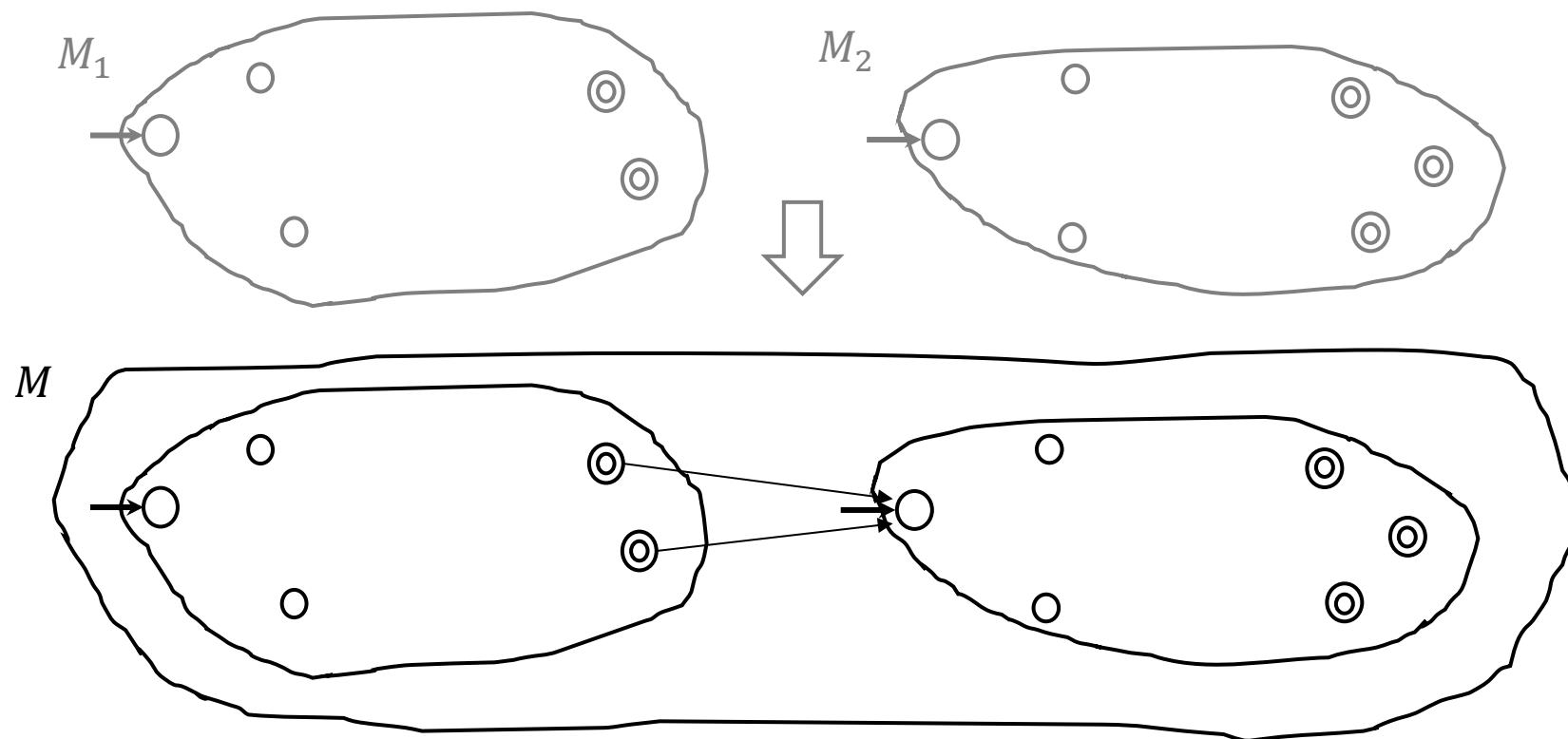
Closure Properties continued

Theorem: If A_1, A_2 are regular languages, so is A_1A_2 (closure under $\circ$)

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1

$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$

Construct $M = (Q, \Sigma, \delta, q_0, F)$

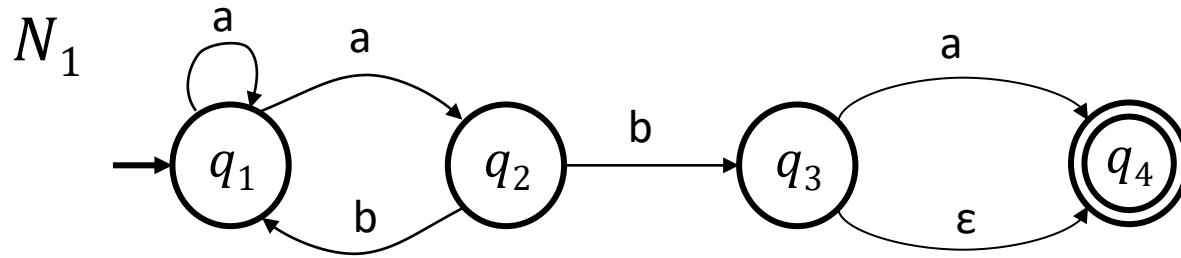


M should accept input w
if $w = xy$ where
 M_1 accepts x and M_2 accepts y .

w ————|—————
 x y

Doesn't work: Where to split w ?

Nondeterministic Finite Automata (NFA)



New features of nondeterminism:

- multiple paths possible (0, 1 or many at each step)
- ϵ -transition is a “free” move without reading input
- Accept input if some path leads to one of the accept states.

N_1 accepts **ab**

$$q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3 \xrightarrow{\epsilon} q_4$$

N_1 accepts **aaba**

$$q_1 \xrightarrow{a} q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3 \xrightarrow{a} q_4$$

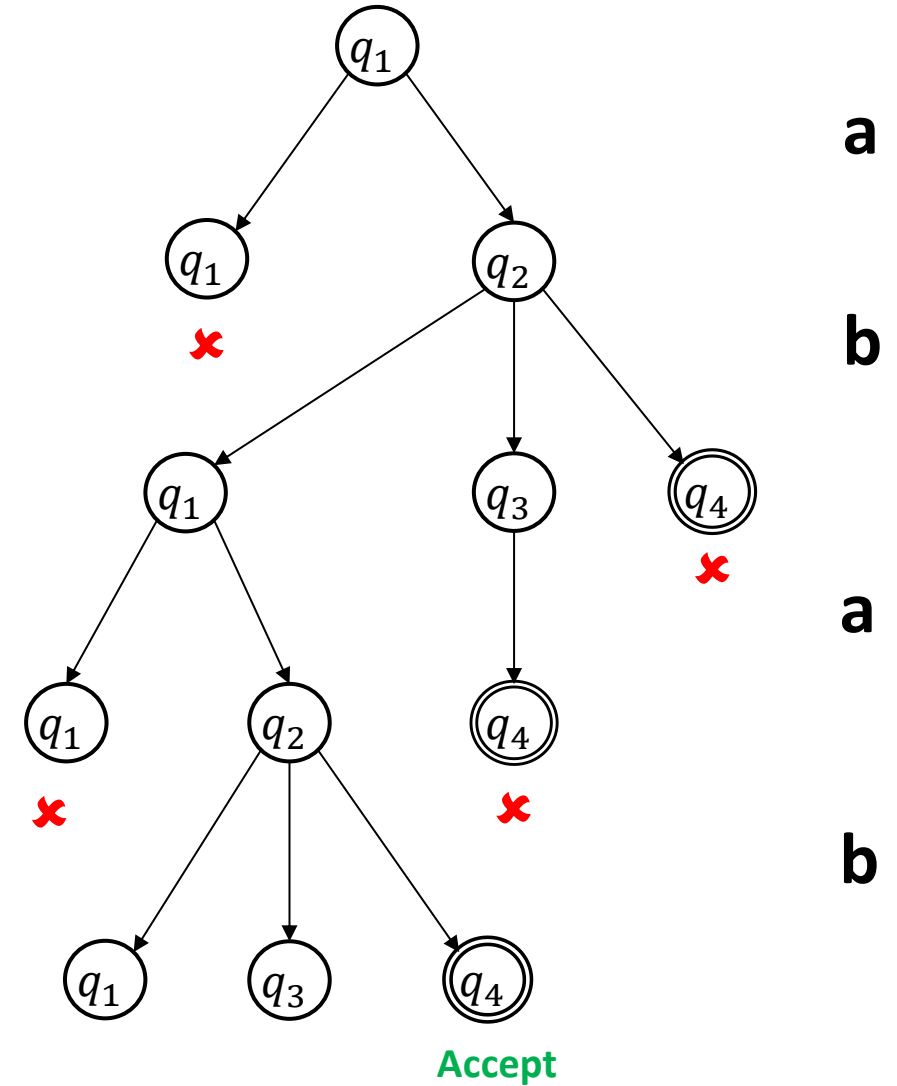
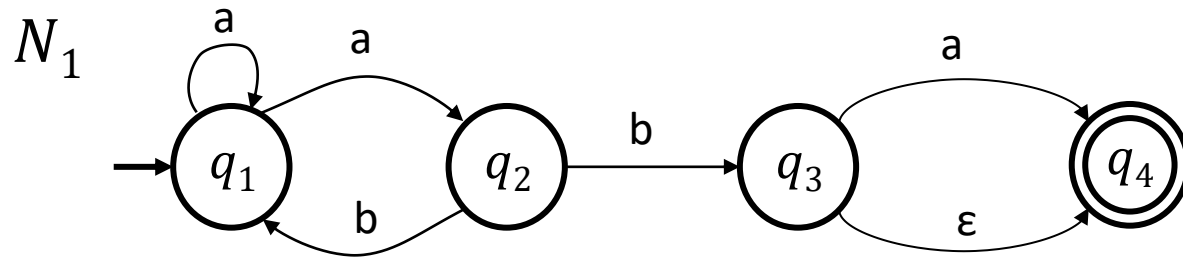
N_1 accepts **abab**

$$q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3 \xrightarrow{\epsilon} q_4$$

N_1 rejects **aa**

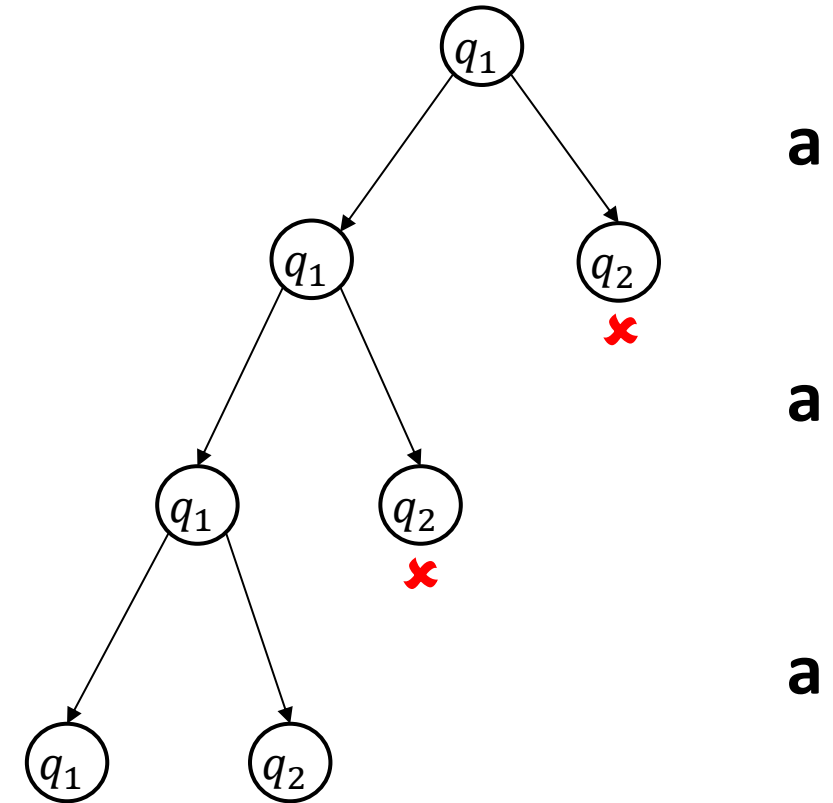
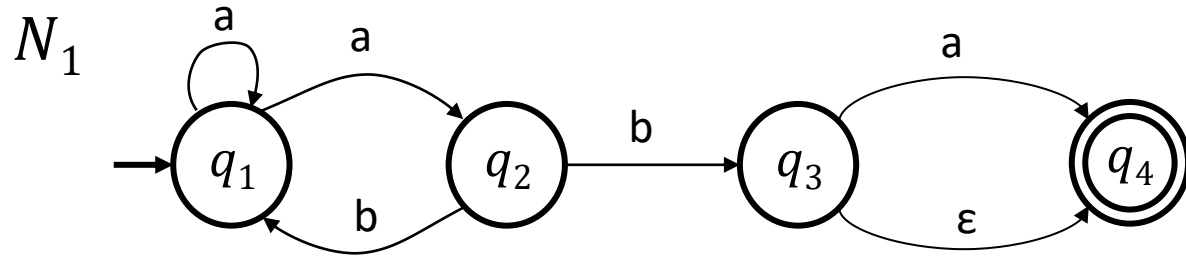
N_1 rejects **abb**

Nondeterministic Finite Automata (NFA)



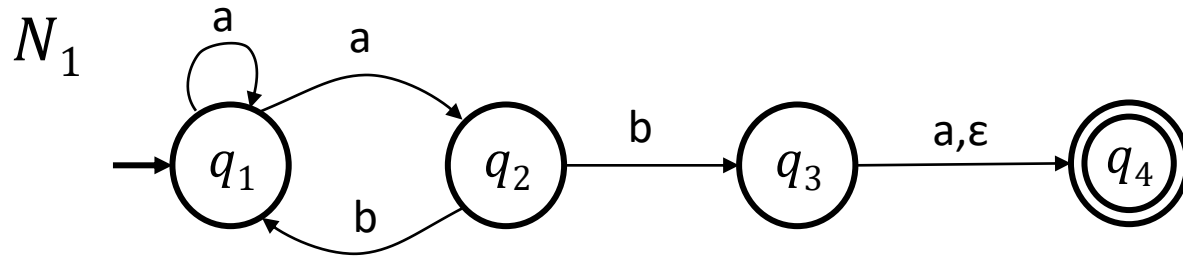
Computation Tree for the string **abab**

Nondeterministic Finite Automata (NFA)



Computation Tree for the string **aaa**

NFA – Formal Definition



Defn: A nondeterministic finite automaton (NFA)

N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

states
alphabet
transition function
start state
accept states

- all same as before except δ
- $\delta: Q \times \underbrace{\Sigma \cup \{\epsilon\}}_{\text{power set}} \rightarrow \mathcal{P}(Q) = \{R | R \subseteq Q\}$
- In the N_1 example: $\delta(q_1, a) = \{q_1, q_2\}$
 $\delta(q_1, b) = \emptyset$

Ways to think about nondeterminism:

Computational: Fork new parallel thread and accept if any thread leads to an accept state.

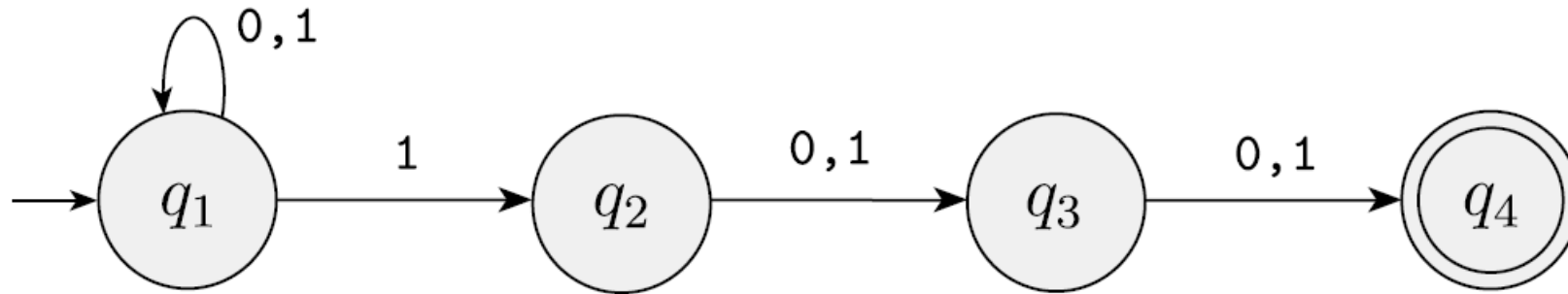
Mathematical: Tree with branches.
Accept if any branch leads to an accept state.

Magical: Guess at each nondeterministic step which way to go. Machine always makes the right guess that leads to accepting, if possible.

Are NFAs more powerful than DFAs?

Let A be the language consisting of all strings over $\{0,1\}$ containing a 1 in the third position from the end (e.g., 000100 is in A but 0011 is not).

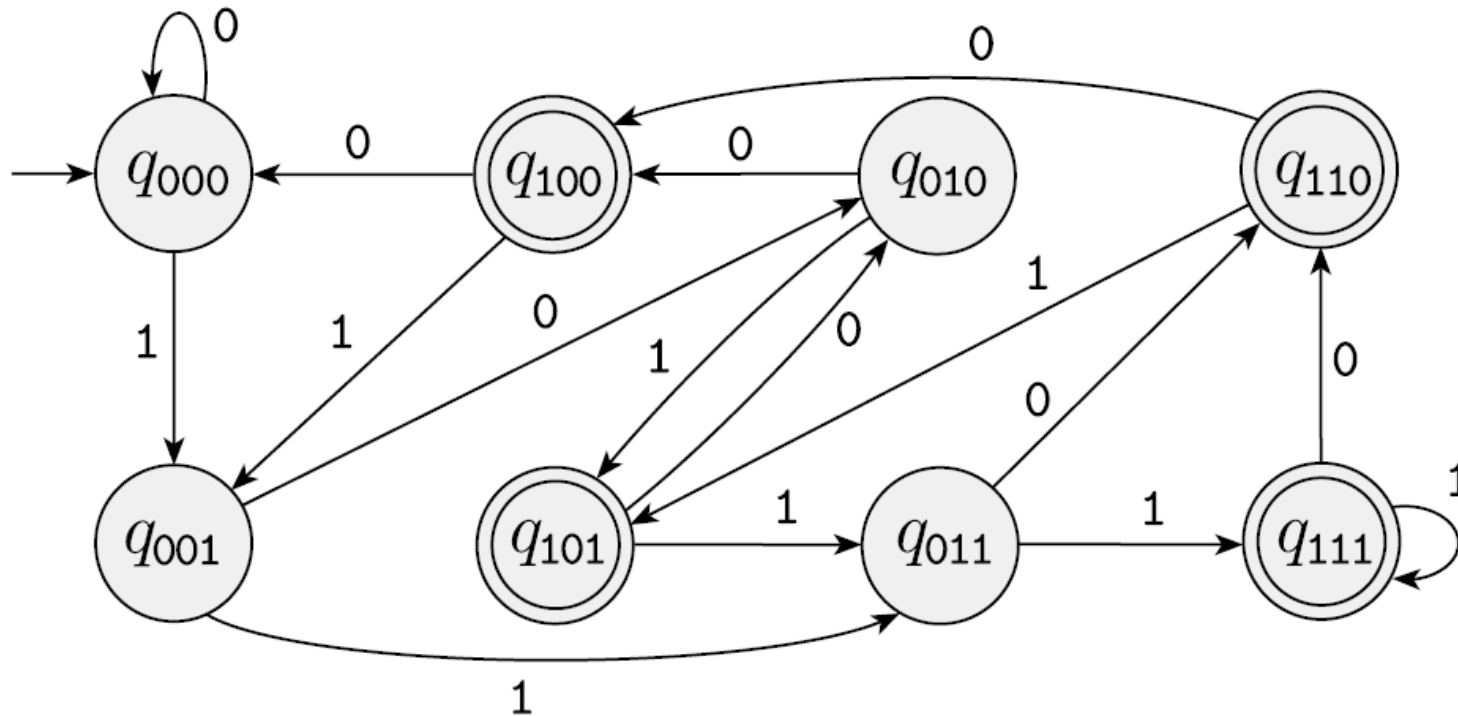
The following four-state NFA N_2 recognizes A .



Is there a deterministic finite automaton recognizing A ?

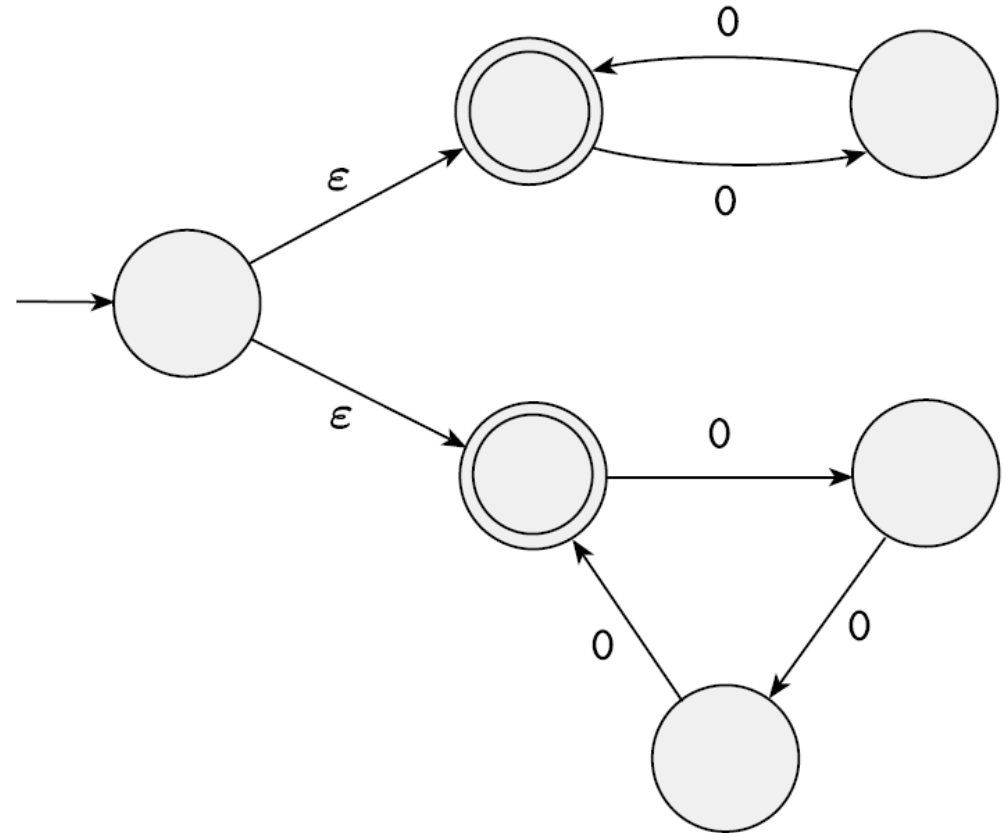
Are NFAs more powerful than DFAs?

Yes! The states in the following DFA remember the last 3 characters read by the automaton.



Are NFAs more powerful than DFAs?

- Suppose $\Sigma = \{0\}$ is the alphabet consisting of only symbol 0.
- The NFA on the right accepts all the strings in which the number of 0 is a multiple of 2 or a multiple of 3.
- Is there an equivalent DFA?



Converting NFAs to DFAs

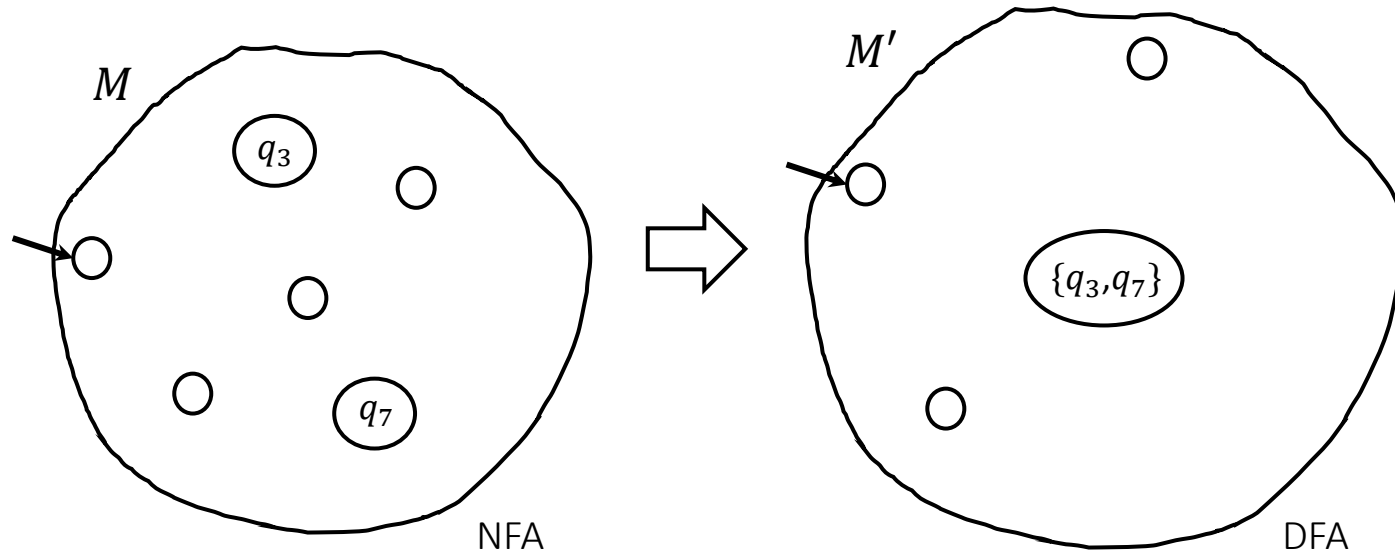
Theorem: If an NFA recognizes A then A is regular

Proof: Let NFA $M = (Q, \Sigma, \delta, q_0, F)$ recognize A
Construct DFA $M' = (Q', \Sigma, \delta', q'_0, F')$ recognizing A

Given a set R of states in M , we define

$$E(R) = \{q \in Q \mid q \in R \text{ or there is an } \varepsilon\text{-path from some state in } R \text{ to } q\}$$

IDEA: DFA M' keeps track of the subset of possible states in NFA M .



Construction of M' :

$$Q' = \mathcal{P}(Q)$$

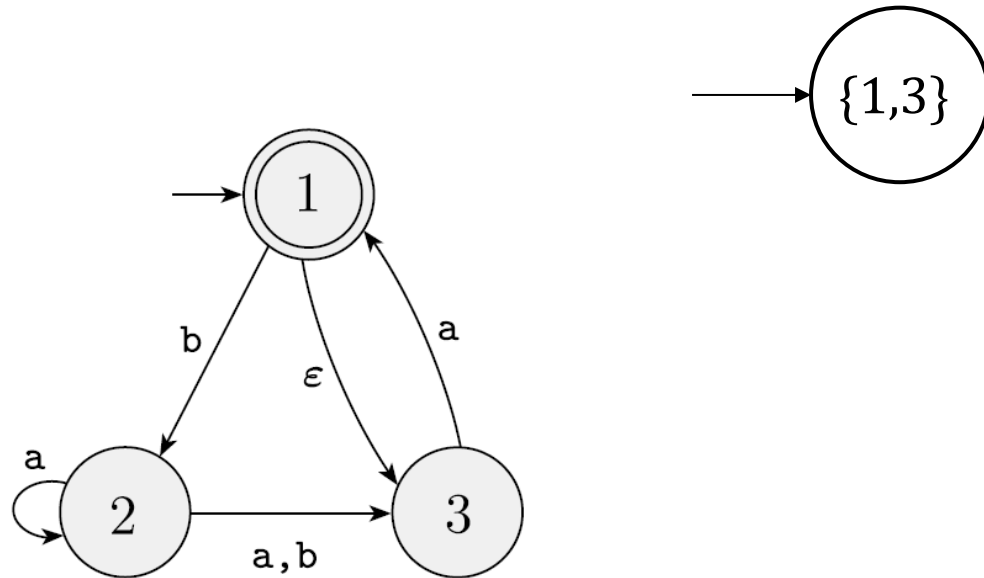
$$\delta'(R, a) = \{q \mid q \in E(\delta(r, a)) \text{ for some } r \in R\}$$

$$q'_0 = E(\{q_0\})$$

$$F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$$

Converting NFAs to DFAs

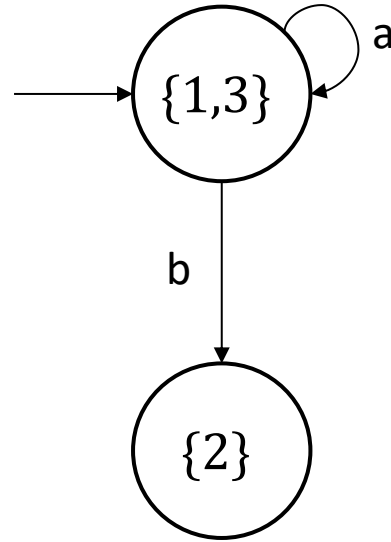
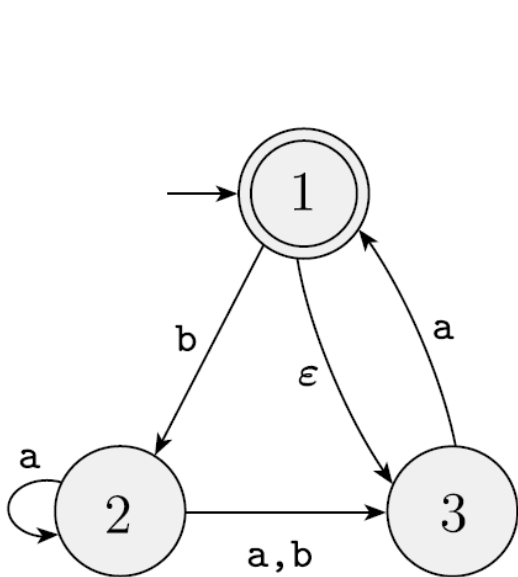
We show how to convert the following NFA into an equivalent DFA.



$$E(\{1\}) = \{1,3\}$$

Converting NFAs to DFAs

We show how to convert the following NFA into an equivalent DFA.

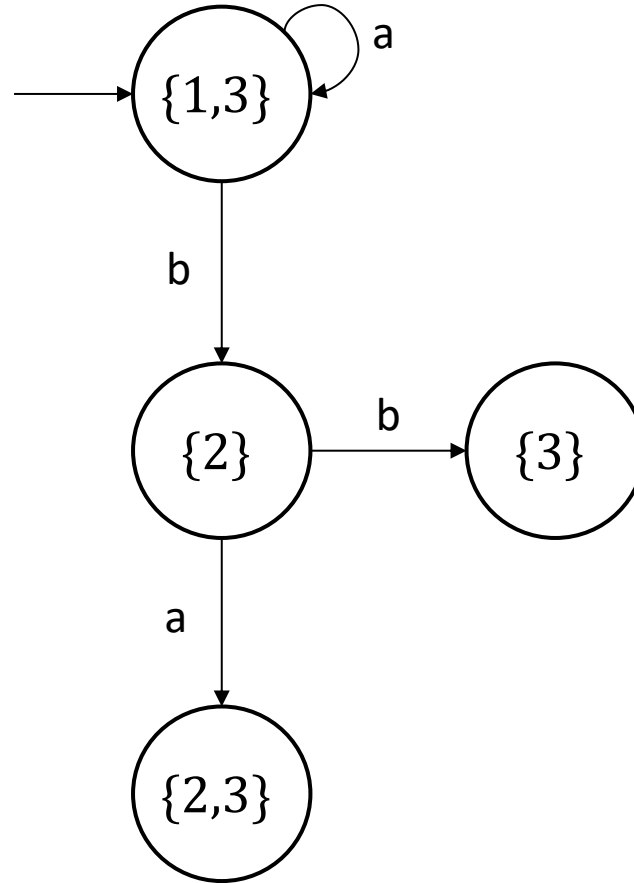
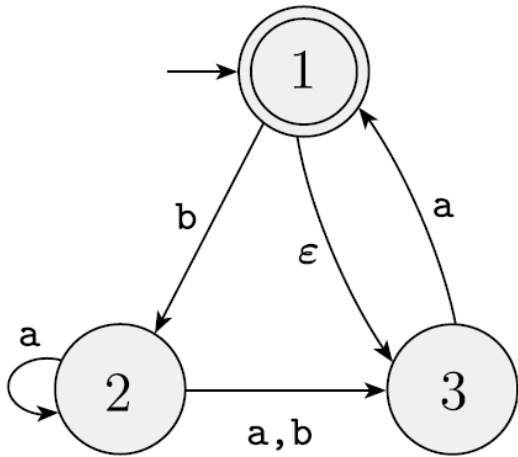


$$\begin{aligned}\delta'(\{1,3\}, a) &= E(\delta(1, a)) \cup E(\delta(3, a)) \\ &= \{ \quad \} \cup \{1,3\}\end{aligned}$$

$$\begin{aligned}\delta'(\{1,3\}, b) &= E(\delta(1, b)) \cup E(\delta(3, b)) \\ &= \{2\} \cup \{ \quad \}\end{aligned}$$

Converting NFAs to DFAs

We show how to convert the following NFA into an equivalent DFA.

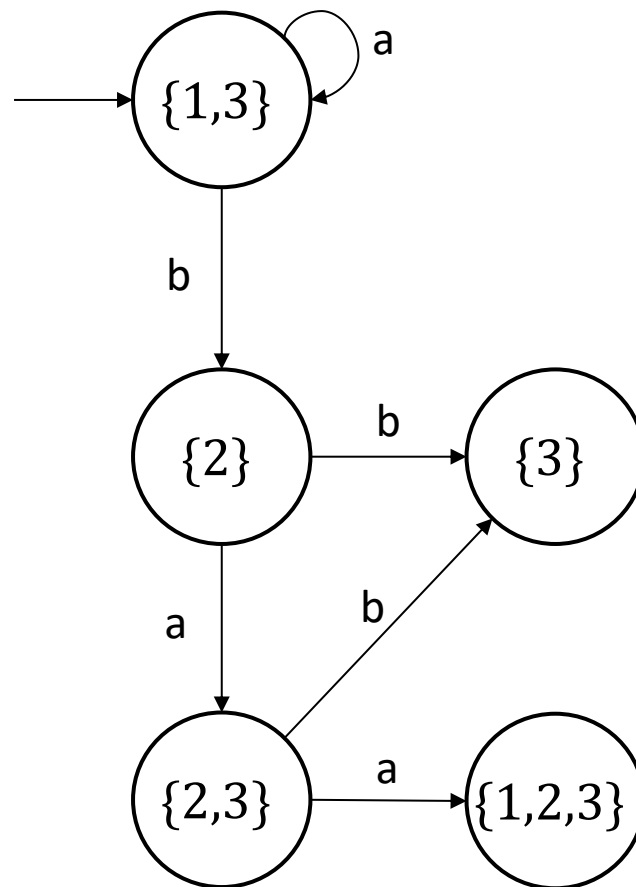
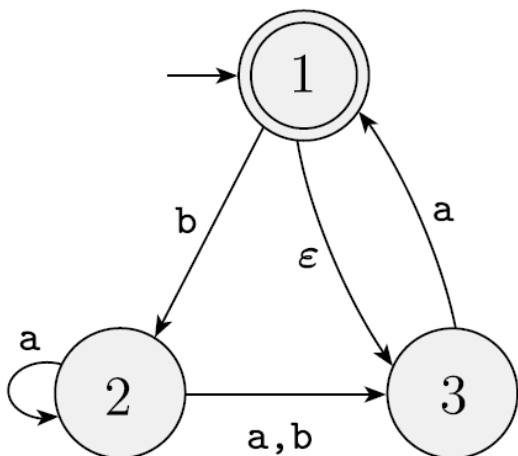


$$\delta'(\{2\}, a) = E(\delta(2, a)) = \{2, 3\}$$

$$\delta'(\{2\}, b) = E(\delta(2, b)) = \{3\}$$

Converting NFAs to DFAs

We show how to convert the following NFA into an equivalent DFA.

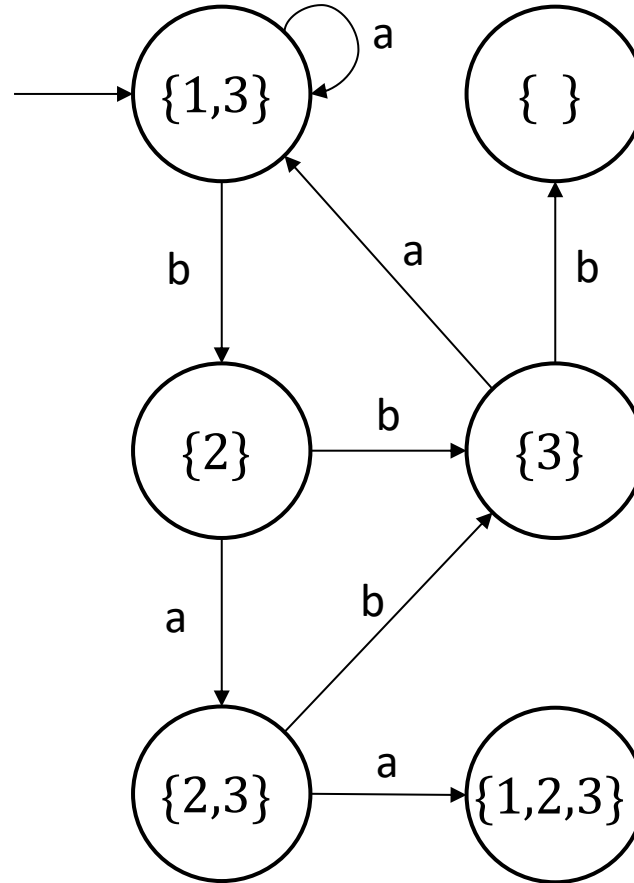
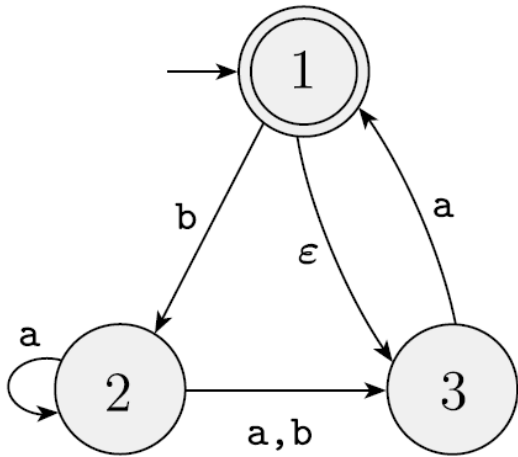


$$\begin{aligned}\delta'(\{2,3\}, a) &= E(\delta(2, a)) \cup E(\delta(3, a)) \\ &= \{2,3\} \cup \{1,3\} \\ &= \{1,2,3\}\end{aligned}$$

$$\begin{aligned}\delta'(\{2,3\}, b) &= E(\delta(2, b)) \cup E(\delta(3, b)) \\ &= \{3\} \cup \{ \} = \{3\}\end{aligned}$$

Converting NFAs to DFAs

We show how to convert the following NFA into an equivalent DFA.

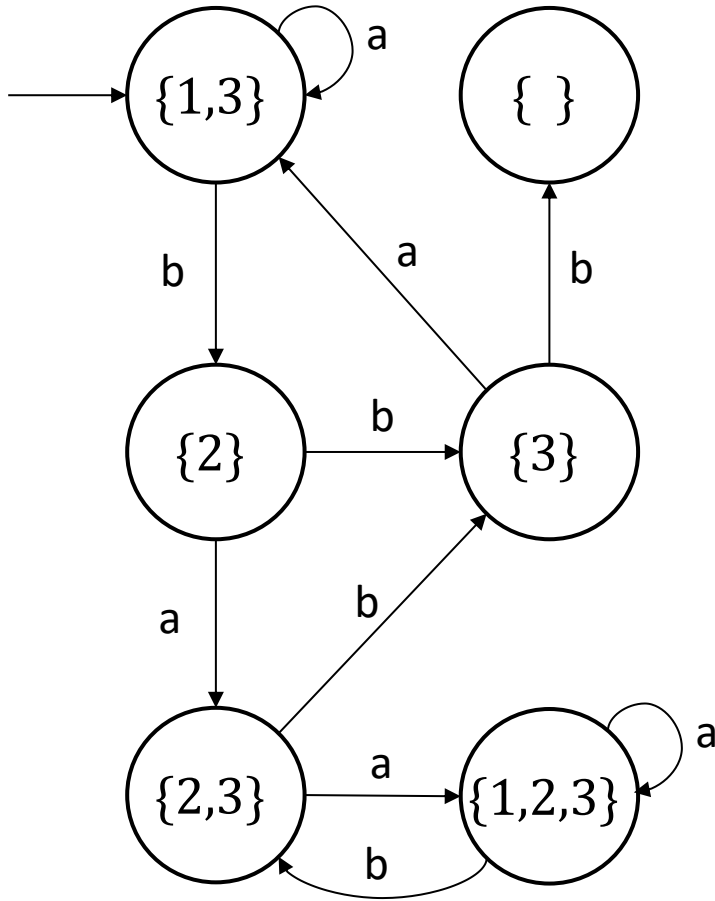
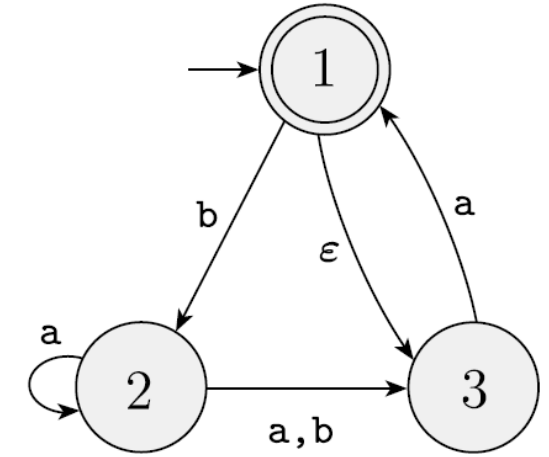


$$\delta'(\{3\}, a) = E(\delta(3, a)) = \{1, 3\}$$

$$\delta'(\{3\}, b) = E(\delta(3, b)) = \{ \}$$

Converting NFAs to DFAs

We show how to convert the following NFA into an equivalent DFA.

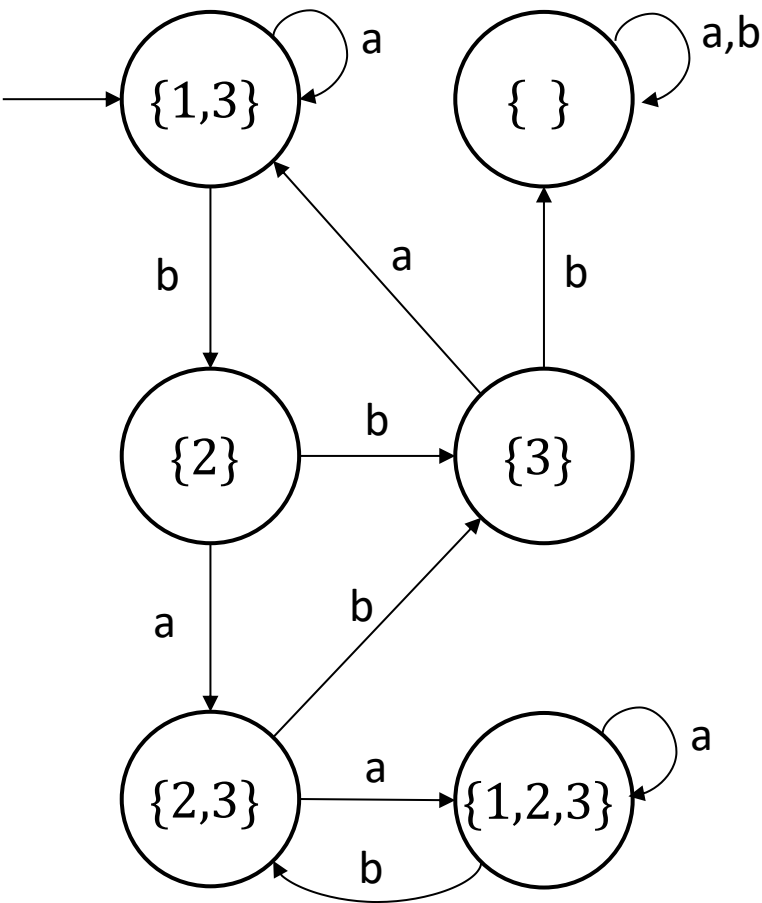
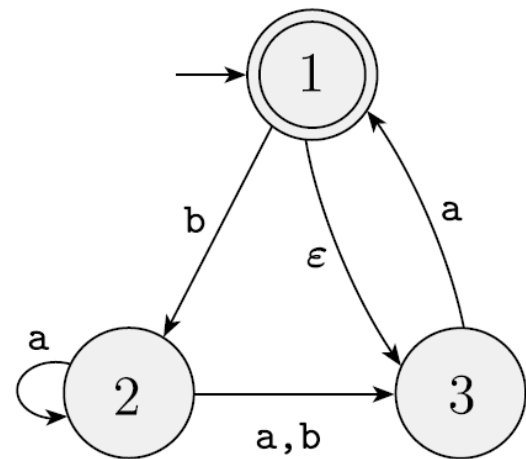


$$\begin{aligned}
 \delta'(\{1,2,3\}, a) &= E(\delta(1, a)) \cup E(\delta(2, a)) \cup E(\delta(3, a)) \\
 &= \{1,3\} \cup \{2,3\} \cup \{1,3\} \\
 &= \{1,2,3\}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(\{1,2,3\}, b) &= E(\delta(1, b)) \cup E(\delta(2, b)) \cup E(\delta(3, b)) \\
 &= \{2\} \cup \{3\} \cup \{2\} \\
 &= \{2,3\}
 \end{aligned}$$

Converting NFAs to DFAs

We show how to convert the following NFA into an equivalent DFA.

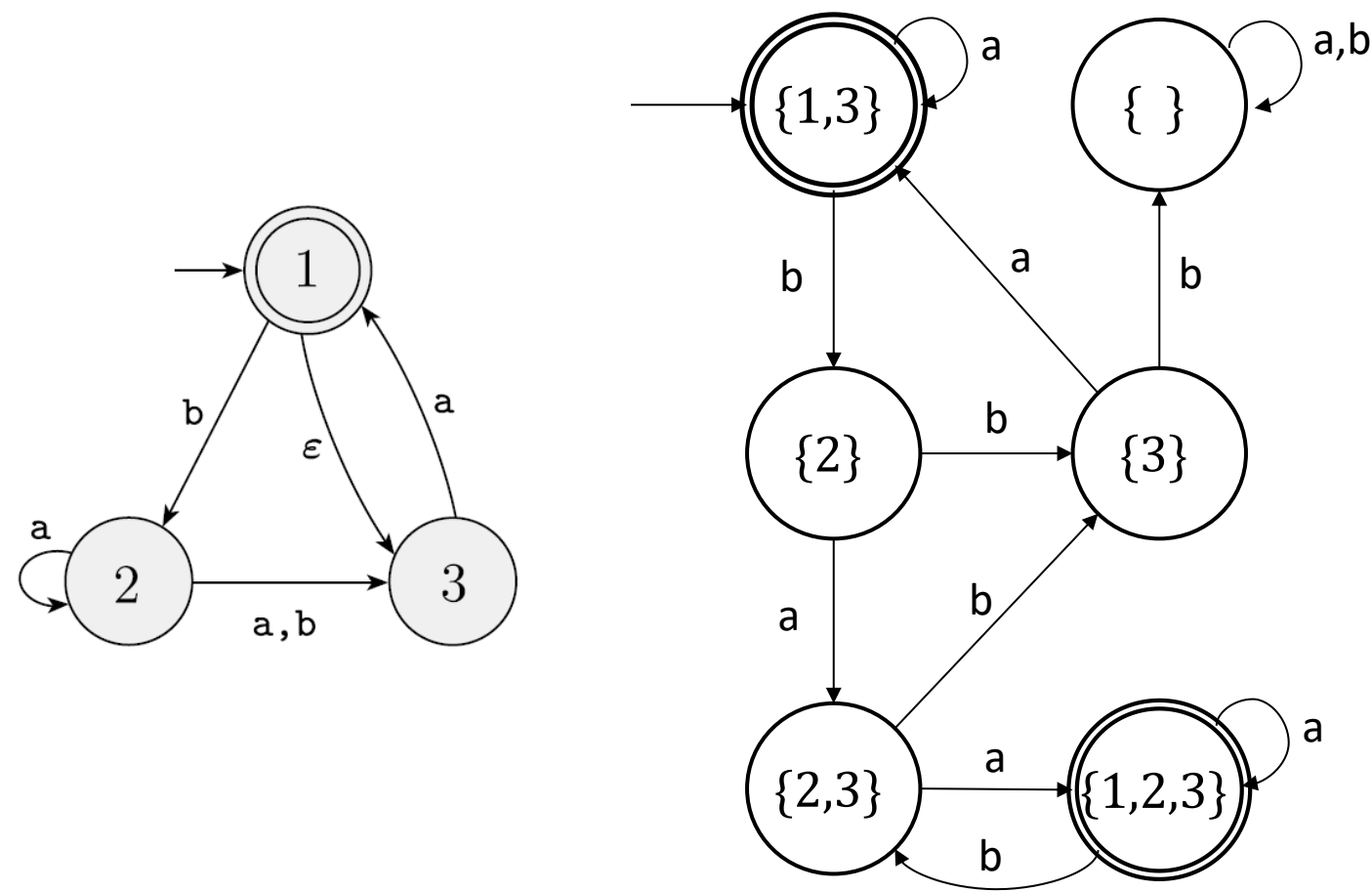


$$\delta'(\{\quad\}, a) = \{\quad\}$$

$$\delta'(\{\quad\}, b) = \{\quad\}$$

Converting NFAs to DFAs

A DFA should accept the given string if and only if the computation ends in a state that contains an accepting state in the original NFA. Hence, we set the accepting states of the DFA accordingly,



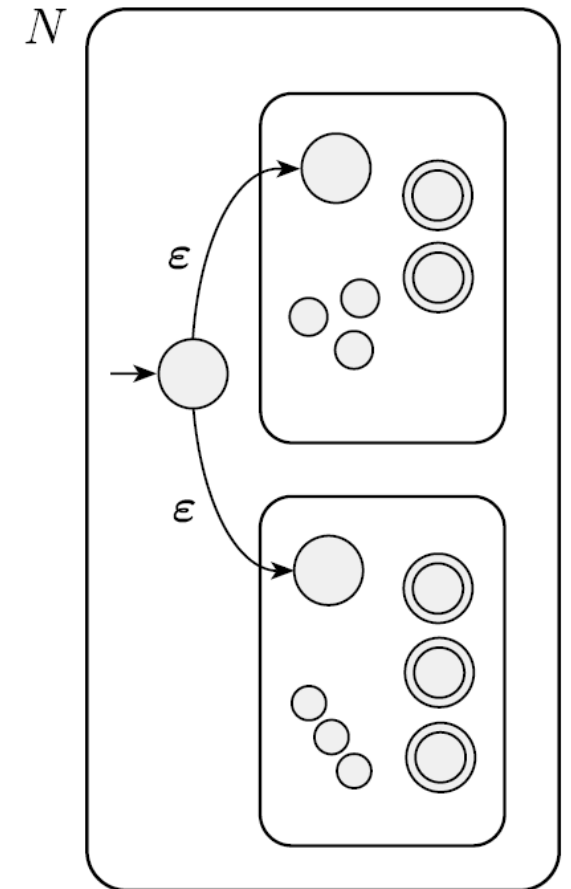
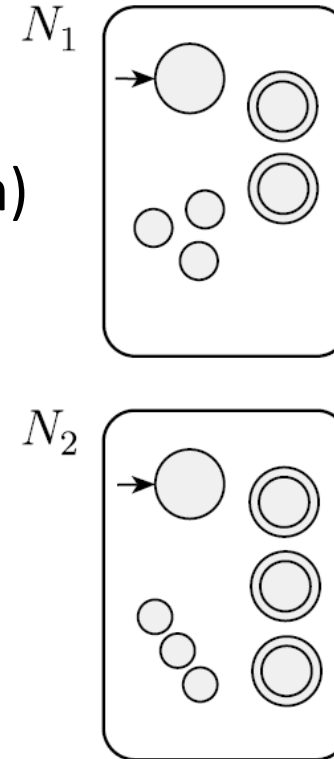
Return to Closure Properties

Theorem:

If A_1, A_2 are regular languages, so is $A_1 \cup A_2$.
(The class of regular languages is closed under union)

New Proof (sketch):

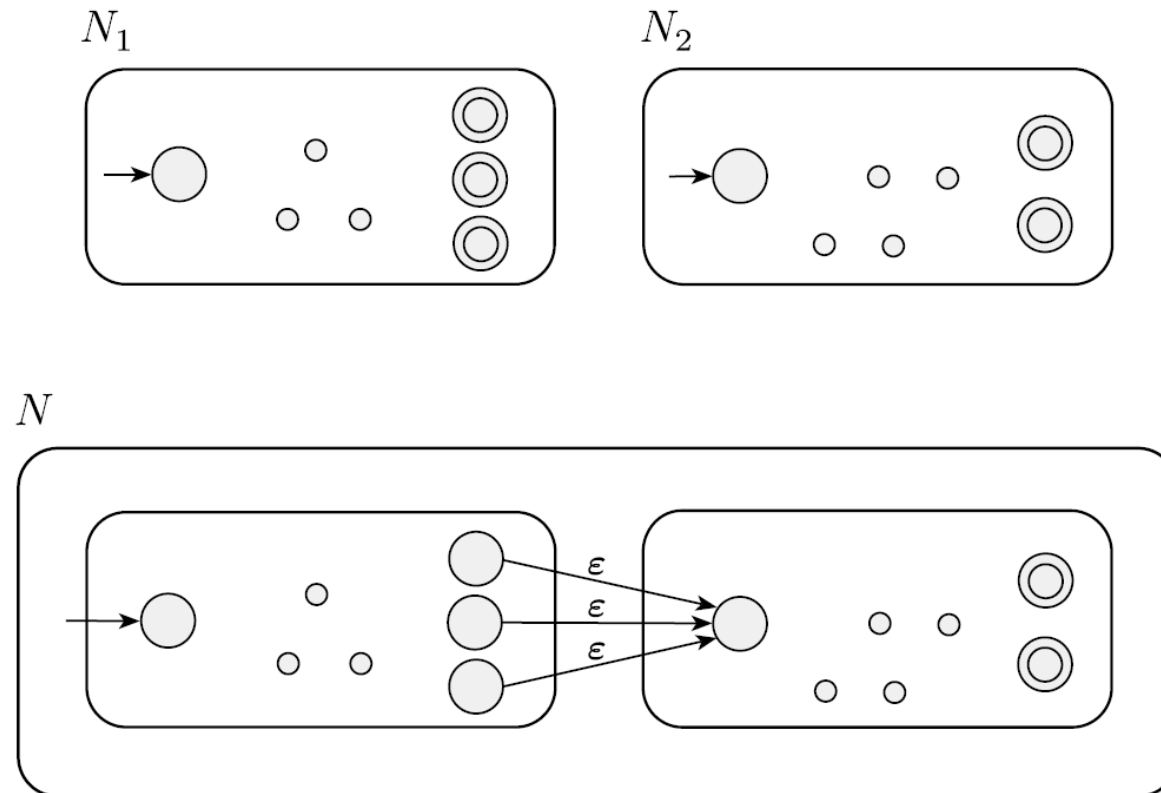
Given DFAs M_1 and M_2 recognizing A_1 and A_2
Construct NFA M recognizing $A_1 \cup A_2$



Closure under $\circ$ (concatenation)

Theorem: If A_1, A_2 are regular languages, so is A_1A_2

Proof sketch: Given DFAs M_1 and M_2 recognizing A_1 and A_2
Construct NFA M recognizing A_1A_2



Closure under $*$ (star)

Theorem: If A is a regular language, so is A^* .

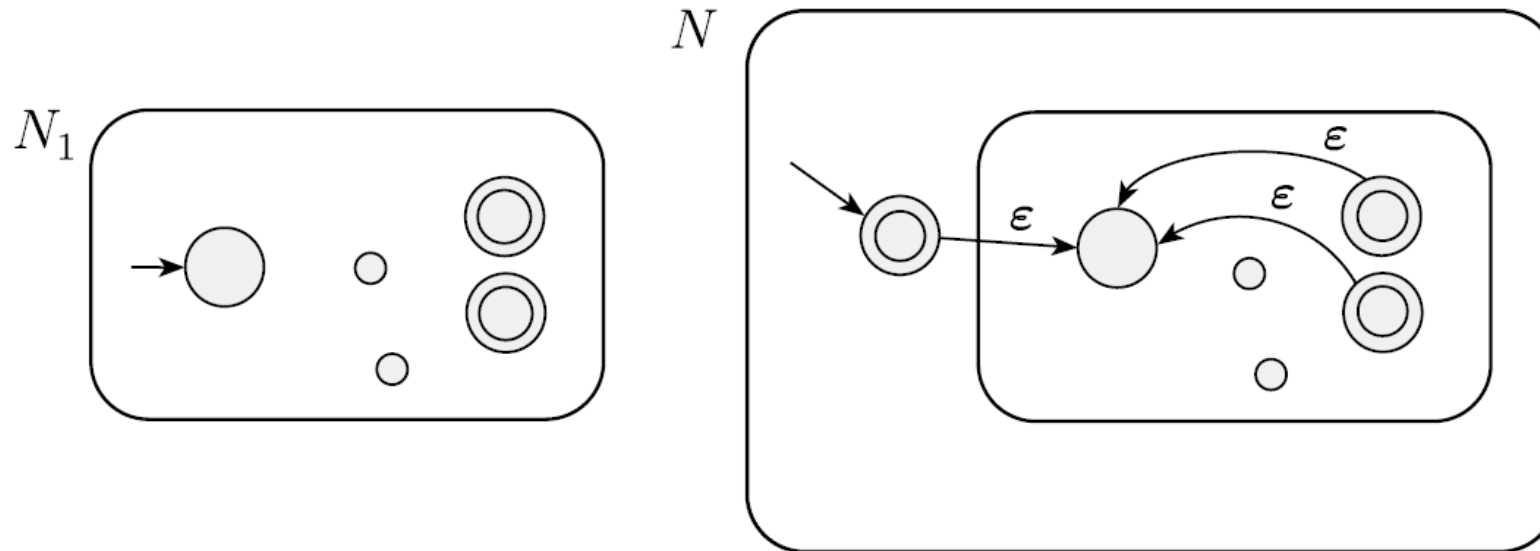
Proof sketch: Given DFA M recognizing A
Construct NFA M' recognizing A^*

M' should accept input w

if $w = x_1x_2 \dots x_k$

where $k \geq 0$ and M accepts each x_i

$w = \begin{array}{c} \text{---} | \text{---} | \text{---} | \text{---} \\ x_1 \quad x_2 \quad x_3 \quad x_4 \end{array}$



Regular Expressions

Regular expressions

- Built from
 - Σ
 - characters in Σ such as 0, 1
 - empty string ε
 - empty language $\emptyset$
- By using the operators: $\cup$, $\circ$, $*$

Examples:

- $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ
- Σ^*1 gives all strings that end with 1
- $\Sigma^*11\Sigma^* =$ all strings that contain 11 $= L(M_1)$

Goal: Show finite automata equivalent to regular expressions

Regular Expressions $\rightarrow$ NFA

Theorem: If R is a regular expression and $A = L(R)$ then A is regular.

Proof: Convert R to equivalent NFA M :

If R is atomic: Equivalent M is:

$R = a$ for $a \in \Sigma$ $\rightarrow \text{---} \bigcirc \xrightarrow{a} \odot$

$R = \varepsilon$ $\rightarrow \odot$

$R = \emptyset$ $\rightarrow \bigcirc$

If R is composite:

$R = R_1 \cup R_2$
 $R = R_1 \circ R_2$
 $R = R_1^*$

} Use closure constructions

Example:

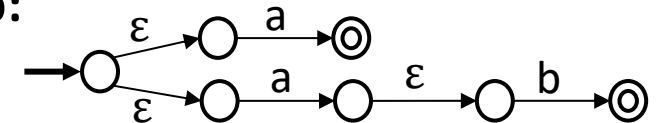
Convert $(a \cup ab)^*$ to equivalent NFA

a : $\rightarrow \bigcirc \xrightarrow{a} \odot$

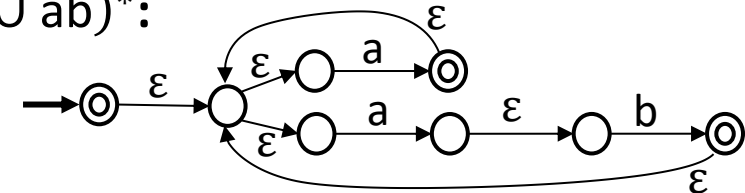
b : $\rightarrow \bigcirc \xrightarrow{b} \odot$

ab : $\rightarrow \bigcirc \xrightarrow{a} \odot \xrightarrow{\varepsilon} \bigcirc \xrightarrow{b} \odot$

$a \cup ab$:



$(a \cup ab)^*$:



DFAs $\rightarrow$ Regular Expressions

Next, let us look at the converse of the previous theorem.

Given conversion DFA M , is there an equivalent regular expression R , i.e. $L(R) = L(M)$?

The answer is Yes!

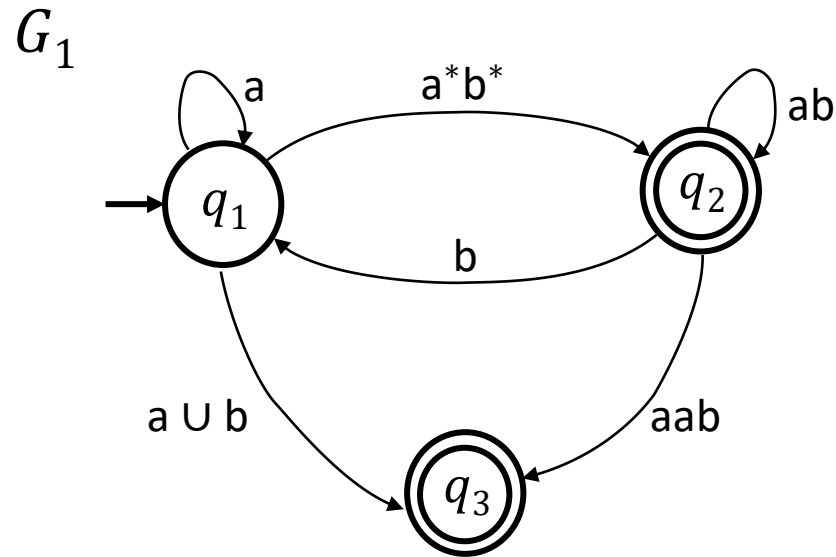
Theorem: If A is regular then $A = L(R)$ for some regular expression R .

Proof: Since A is a regular language, by definition, there exists a DFA M such that $L(M) = A$. We can show how to write an equivalent regular expression R from DFA M .

One way is to subsequently reducing the number of states in M by converting it into a more general kind of automaton, which is called a Generalized Nondeterministic Finite Automaton (GNFA) until we obtain a GNFA with only two states.

Generalized NFA

Def: A Generalized Nondeterministic Finite Automaton (GNFA) is similar to an NFA but allows regular expressions as transition labels.

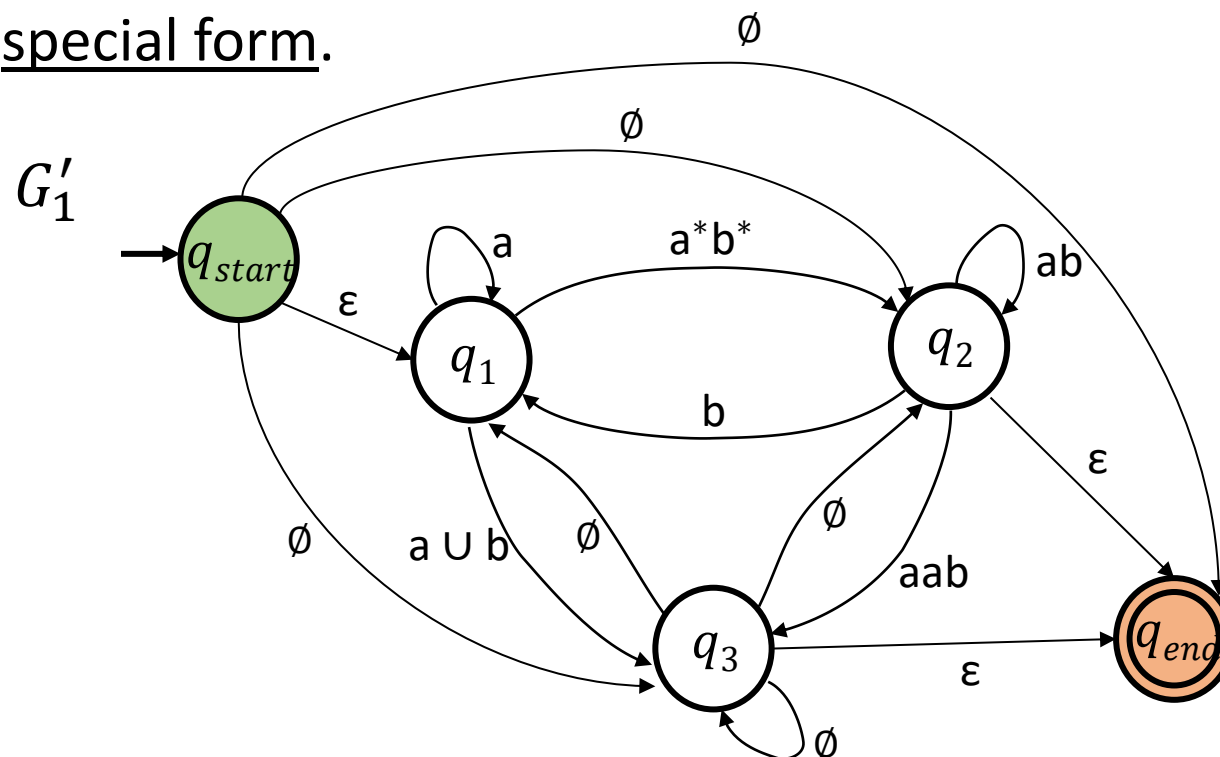
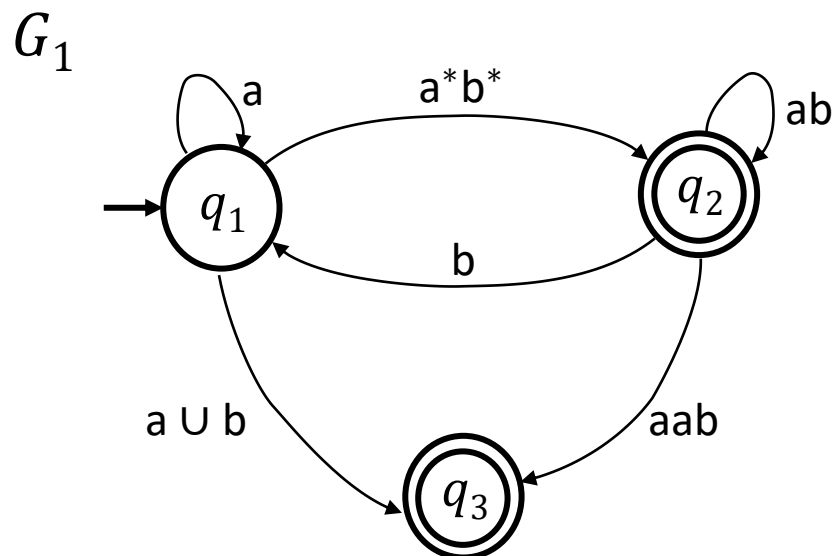


Generalized NFA in special form

For convenience we will assume:

- One accept state, separate from the start state
- One arrow from each state to each state, except
 - a) only exiting the start state
 - b) only entering the accept state

We can easily modify a GNFA to have this special form.



GNFA $\rightarrow$ Regular Expressions

Lemma: Every GNFA G has an equivalent regular expression R .

Proof: By induction on the number of states k of G

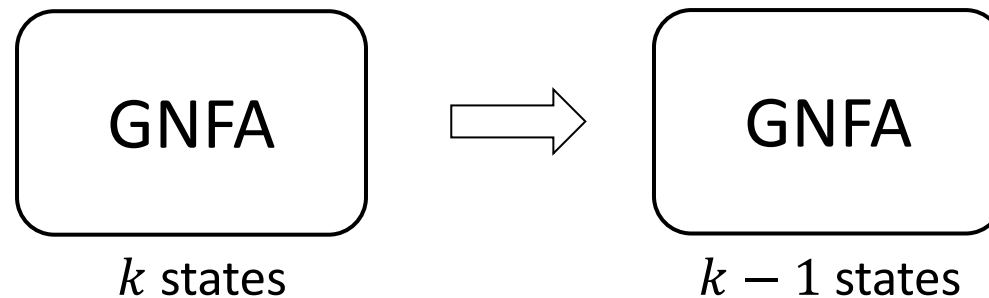
Basis ($k = 2$):

$G = \rightarrow \bigcirc \xrightarrow{r} \odot$ Remember: G is in special form

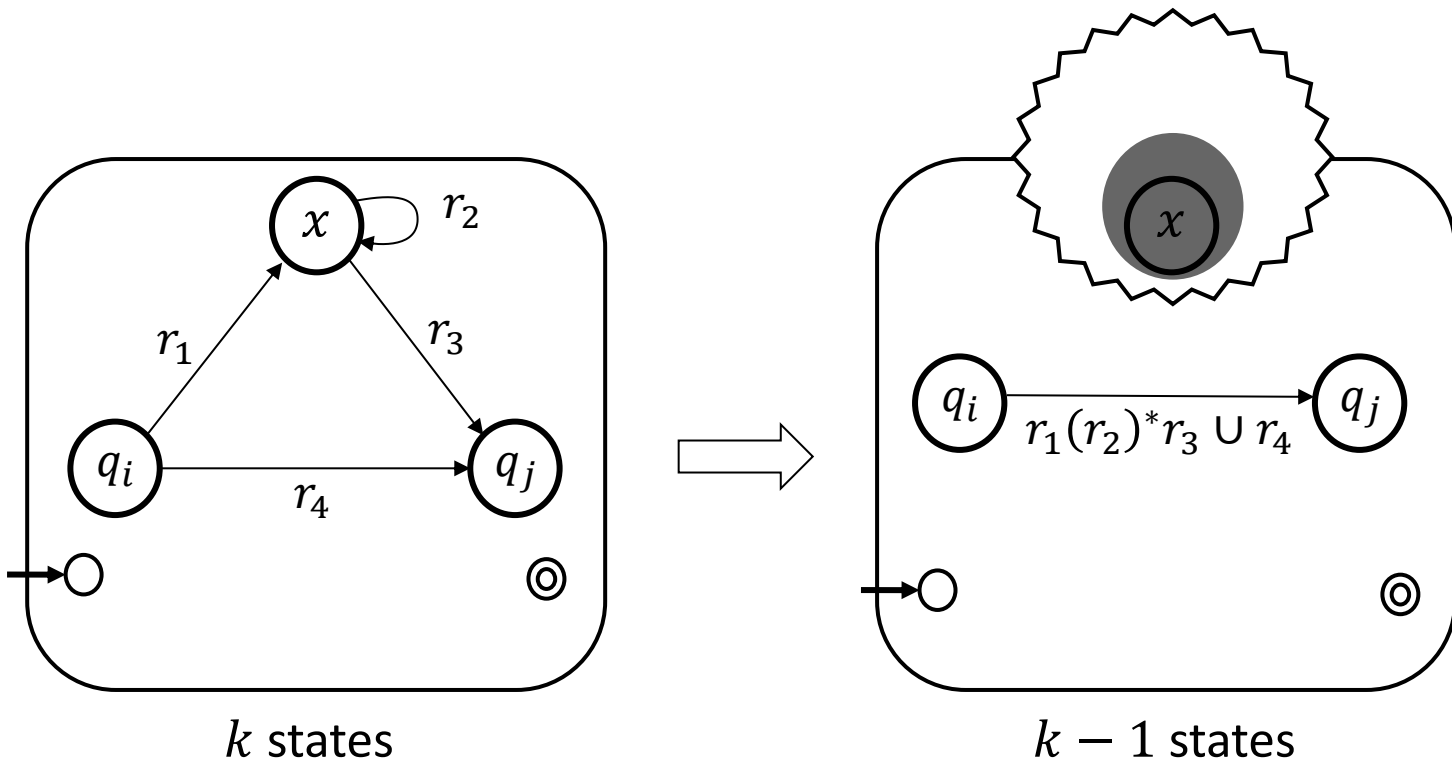
Let $R = r$

Induction step ($k > 2$): Assume Lemma true for $k - 1$ states and prove for k states

IDEA: Convert k -state GNFA to equivalent $(k - 1)$ -state GNFA.

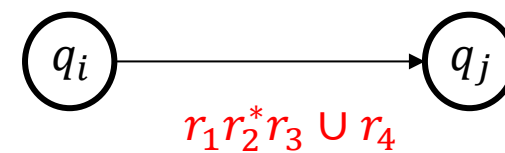
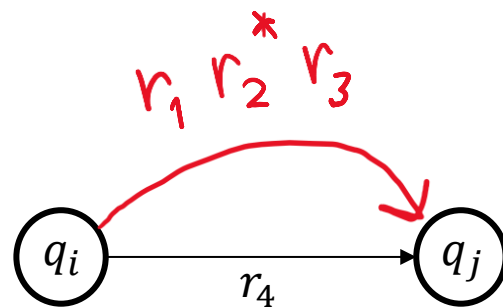
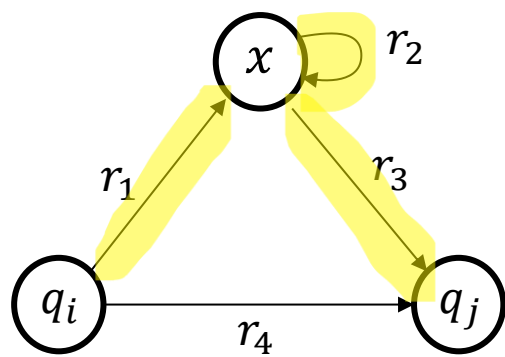


k -state GNFA $\rightarrow (k-1)$ -state GNFA



1. Pick any state x except the start and accept states.
2. Remove x .
3. Repair the damage by recovering all paths that went through x .
4. Make the indicated change for each pair of states q_i, q_j .

Thus, DFAs and regular expressions are equivalent.

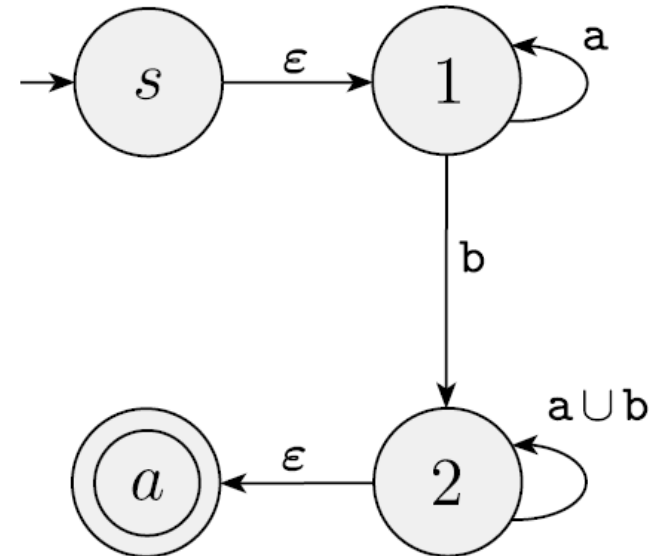
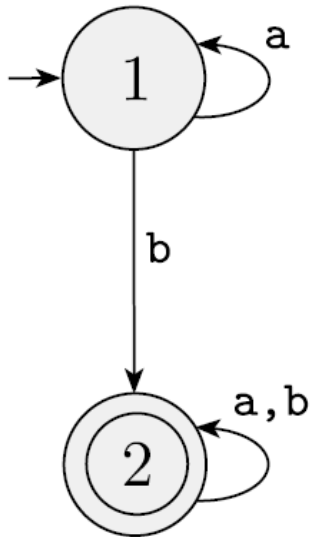


NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

Example 1. Let us look at an example of a conversion from an NFA to an equivalent regular expression.

Step 1: Convert NFA into an GNFA in special form.

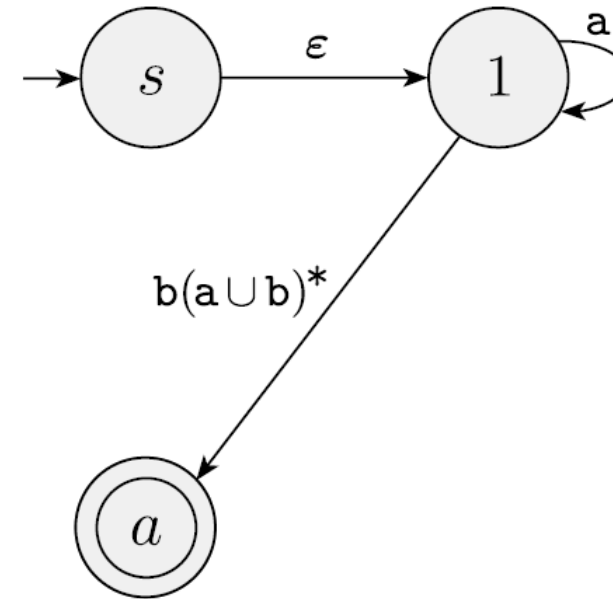
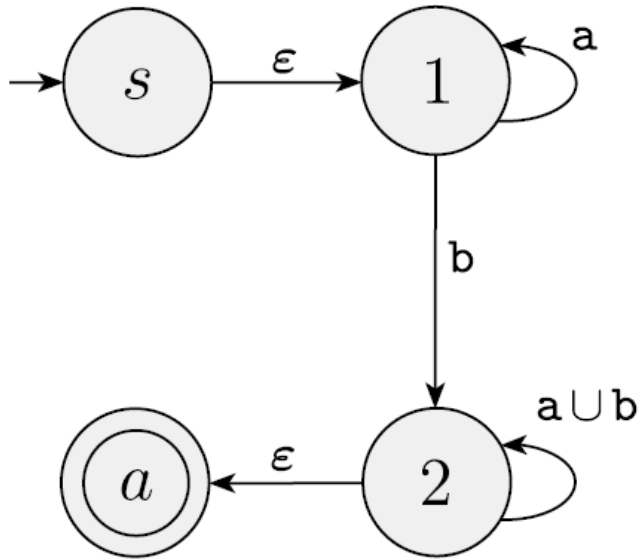
- We add need start state s and new accept state a .
- To avoid clutter, we omit the edge labelled with $\emptyset$.
- We combine the two loops (for a and b) at State 2 into a single loop labelled with $a \cup b$.



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

Step 2: Next, we pick one node other than s and a and remove it.

Here, we first remove State 2.

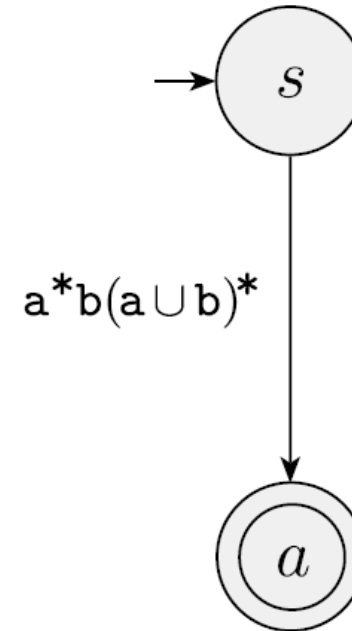
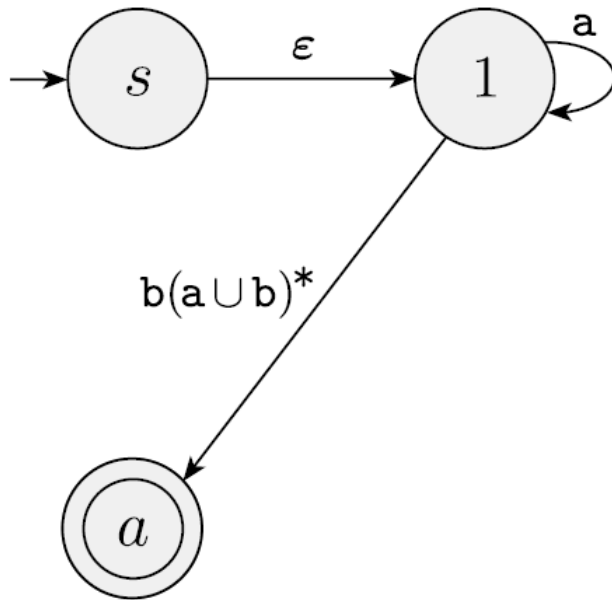


NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

Step 3: Remove State 1.

We obtain a GNFA with only the start state and accept state and a single transition.

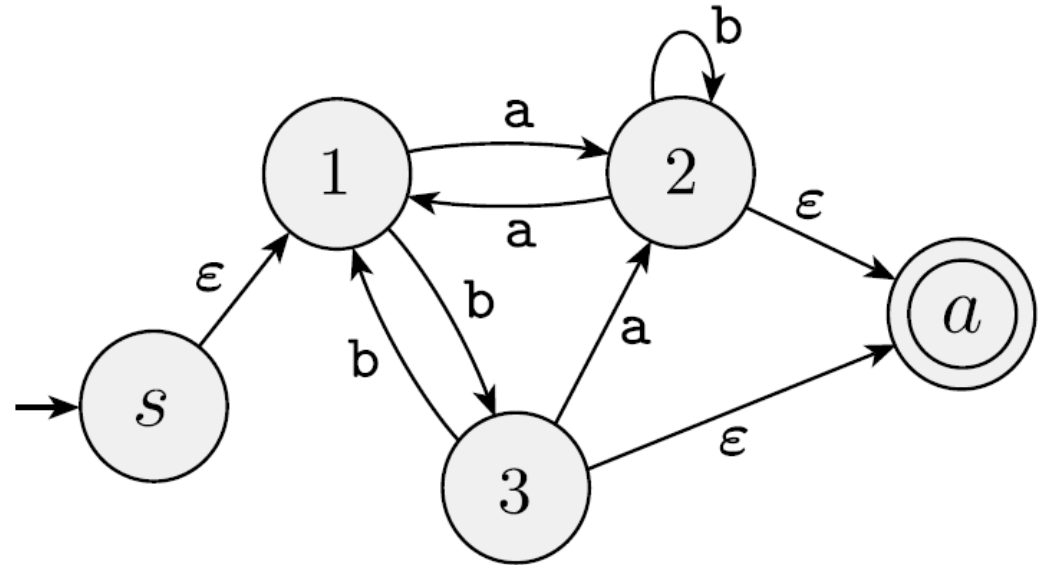
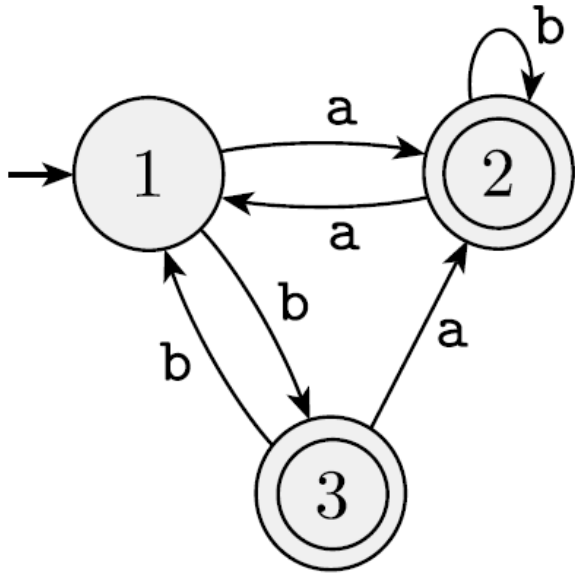
We can then conclude that the given NFA is equivalent to the expression $a^*b(a \cup b)^*$.



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

Example 2. Let us try converting a more complicated NFA.

Step 1: Convert NFA into an GNFA in special form.



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

Step 2: Remove State 1. State 1 currently acts as a bridge connecting between the following pairs of states:

1) $s \rightarrow 1 \rightarrow 2$

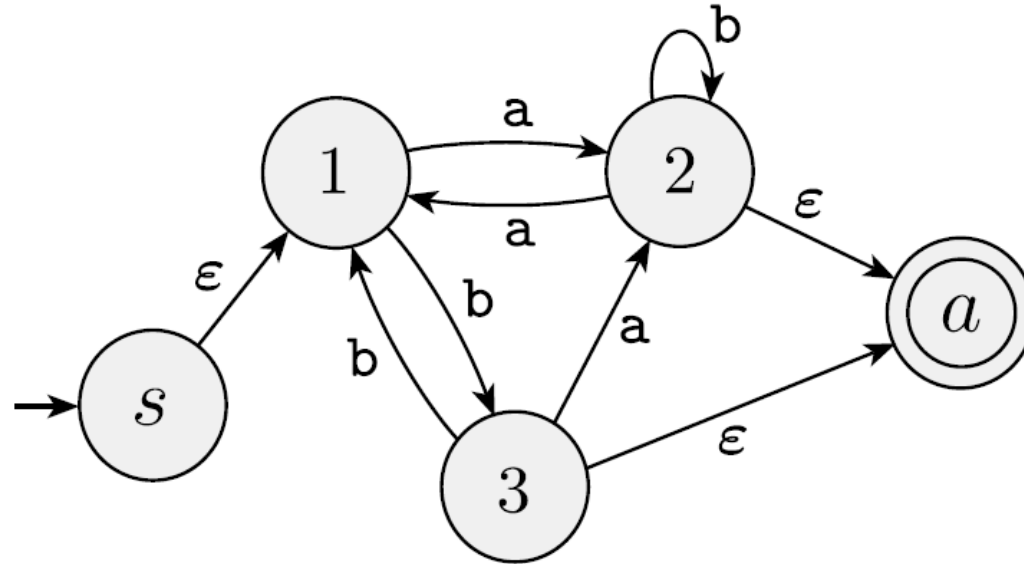
2) $s \rightarrow 1 \rightarrow 3$

3) $2 \rightarrow 1 \rightarrow 3$

4) $2 \rightarrow 1 \rightarrow 2$

5) $3 \rightarrow 1 \rightarrow 2$

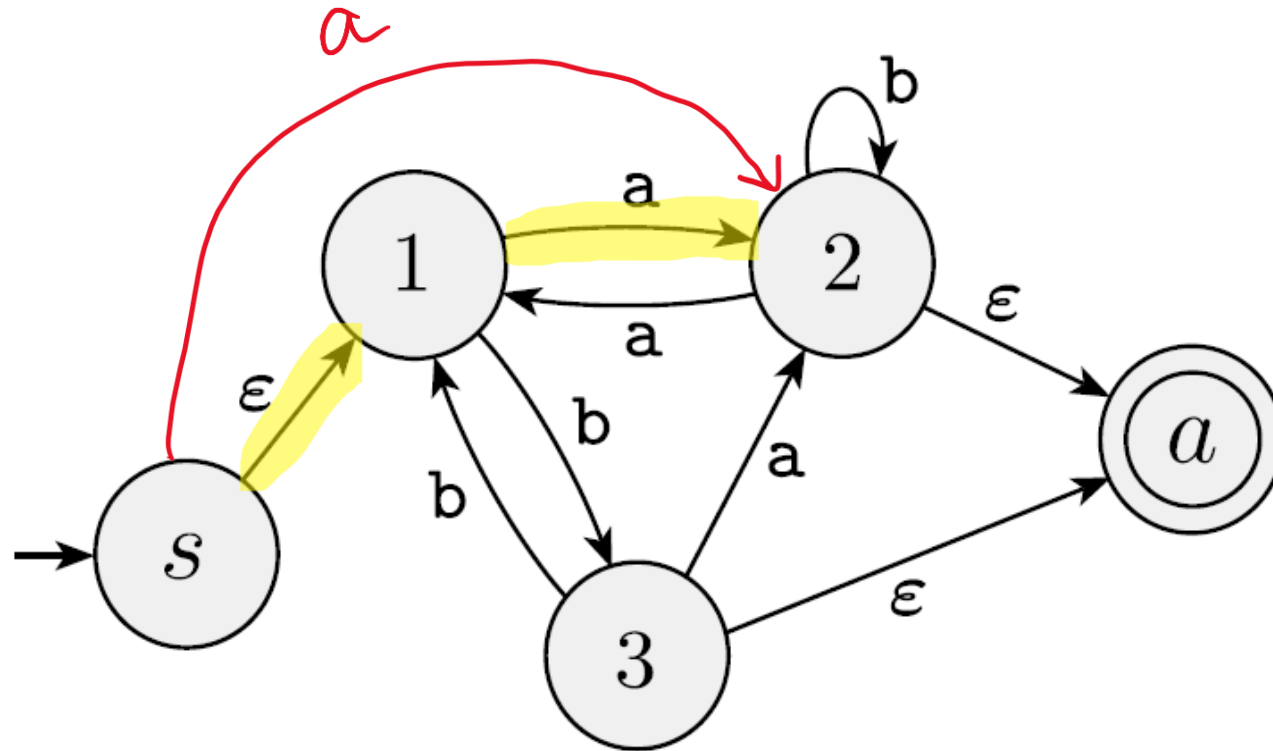
6) $3 \rightarrow 1 \rightarrow 3$



To safely remove State 1, we need to add alternative transitions that directly connect states s , 2 , 3 without going through state 1.

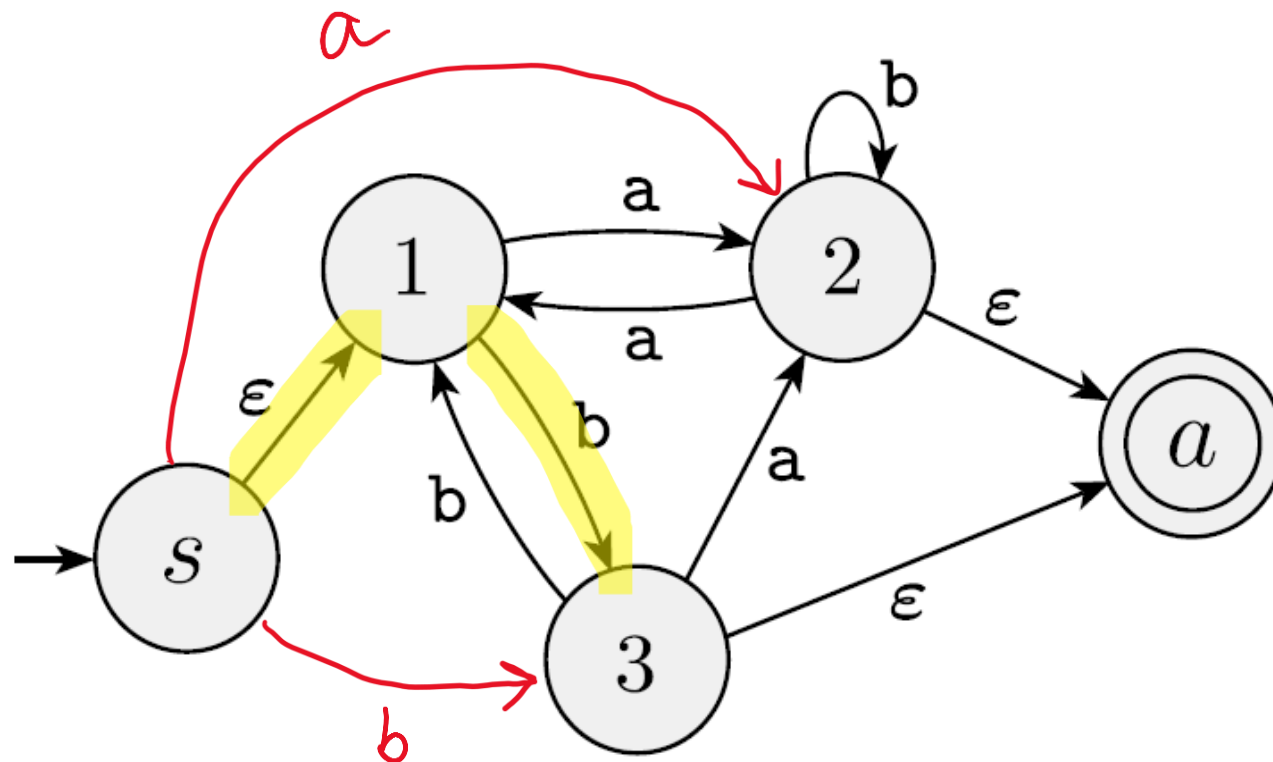
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

$s \rightarrow 1 \rightarrow 2$



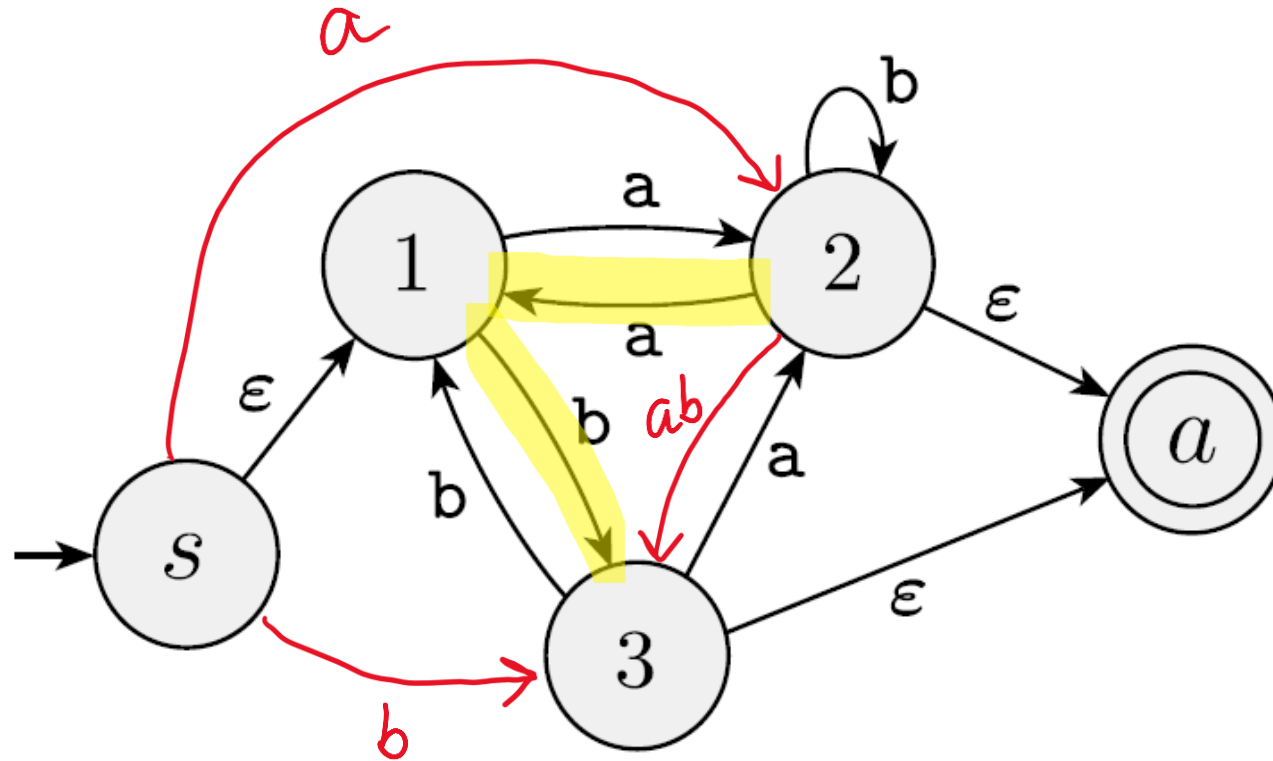
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

$s \rightarrow 1 \rightarrow 3$



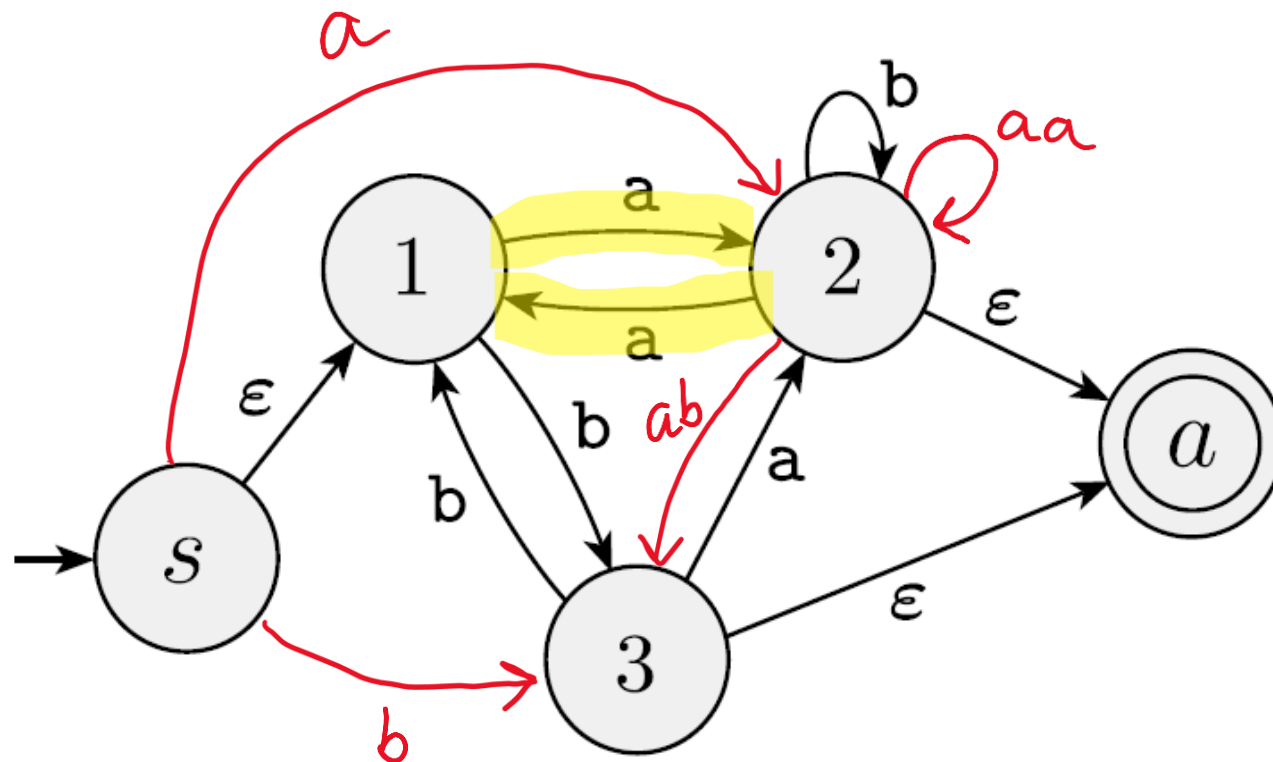
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

2 $\rightarrow$ 1 $\rightarrow$ 3



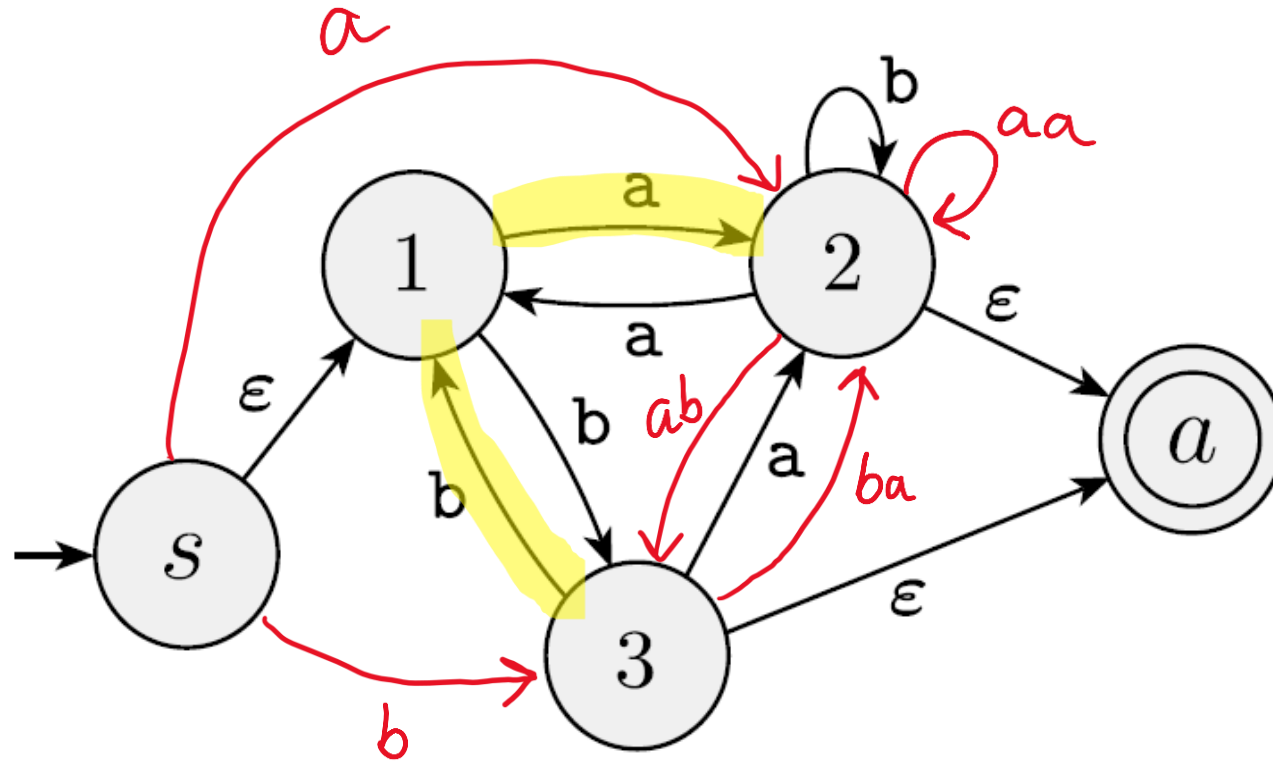
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

2 $\rightarrow$ 1 $\rightarrow$ 2



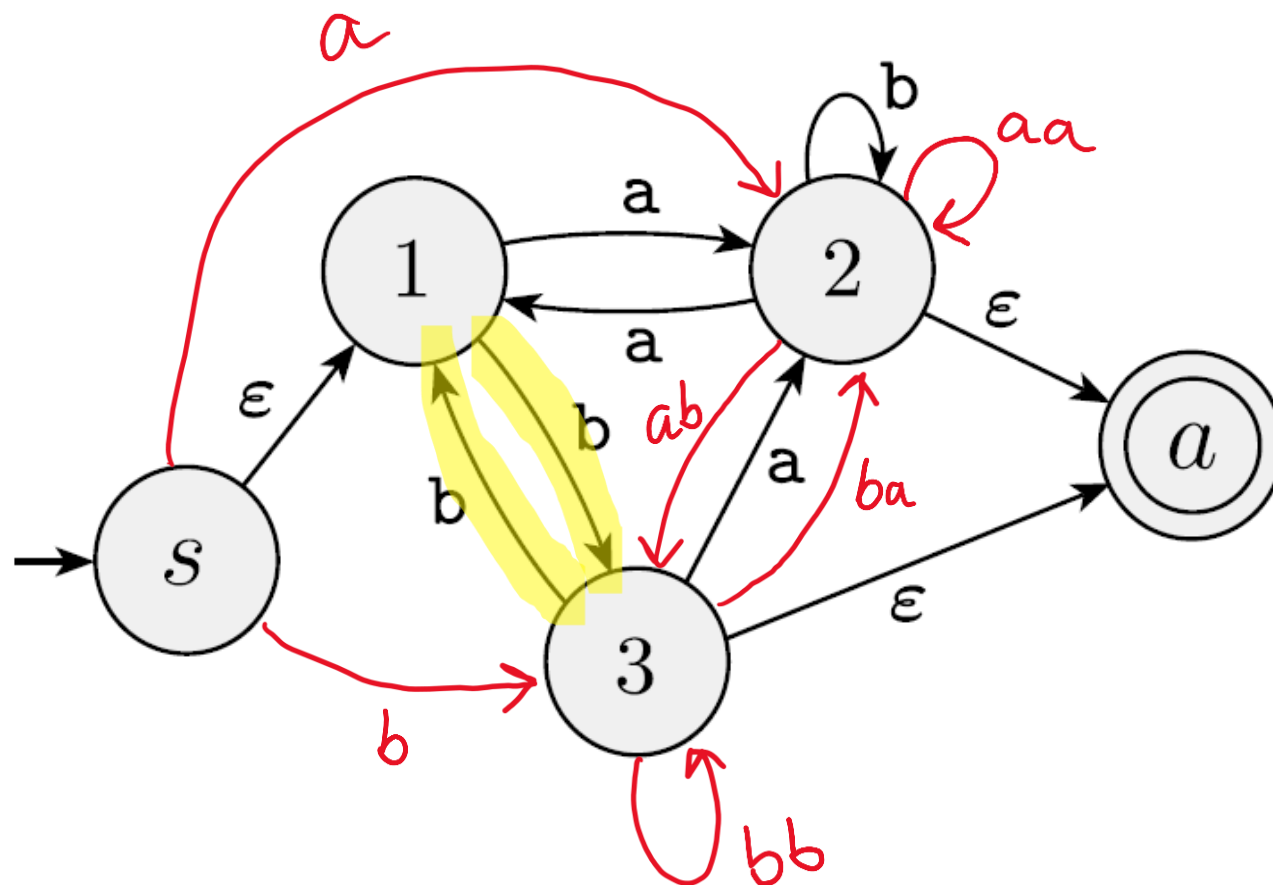
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

3 $\rightarrow$ 1 $\rightarrow$ 2



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

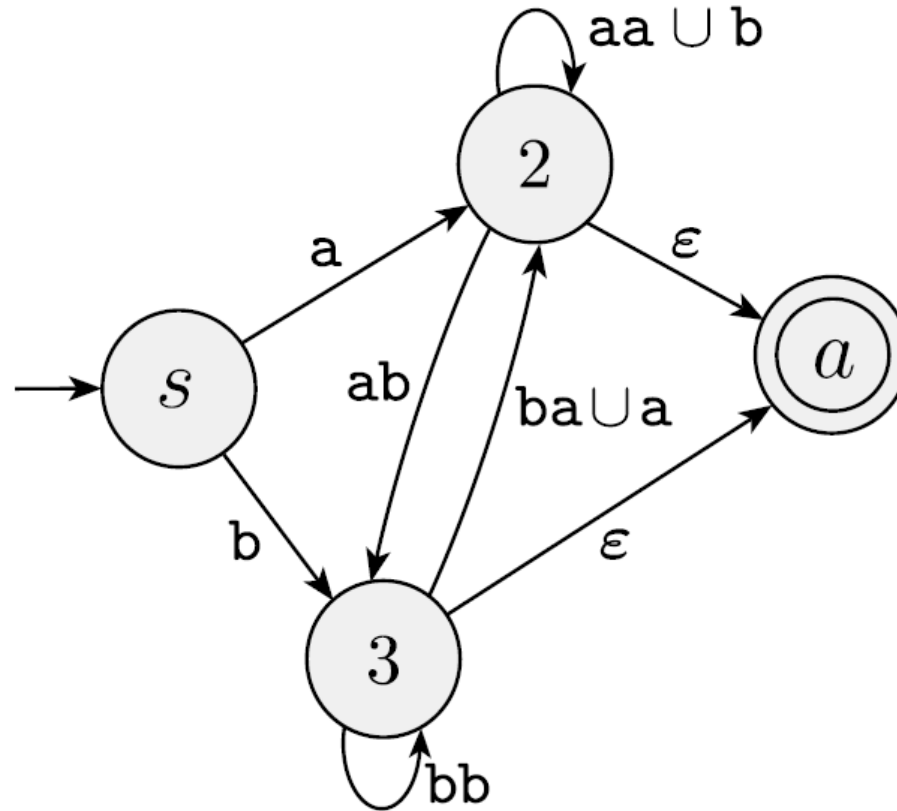
$3 \rightarrow 1 \rightarrow 3$



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

After adding the alternative transitions, we can safely remove state 1.

Note that we combine two transitions with the same origin and the same destination into a single transition by unioning their labels.



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

Step 3: Remove State 2. State 2 currently acts as a bridge connecting between the following pairs of states:

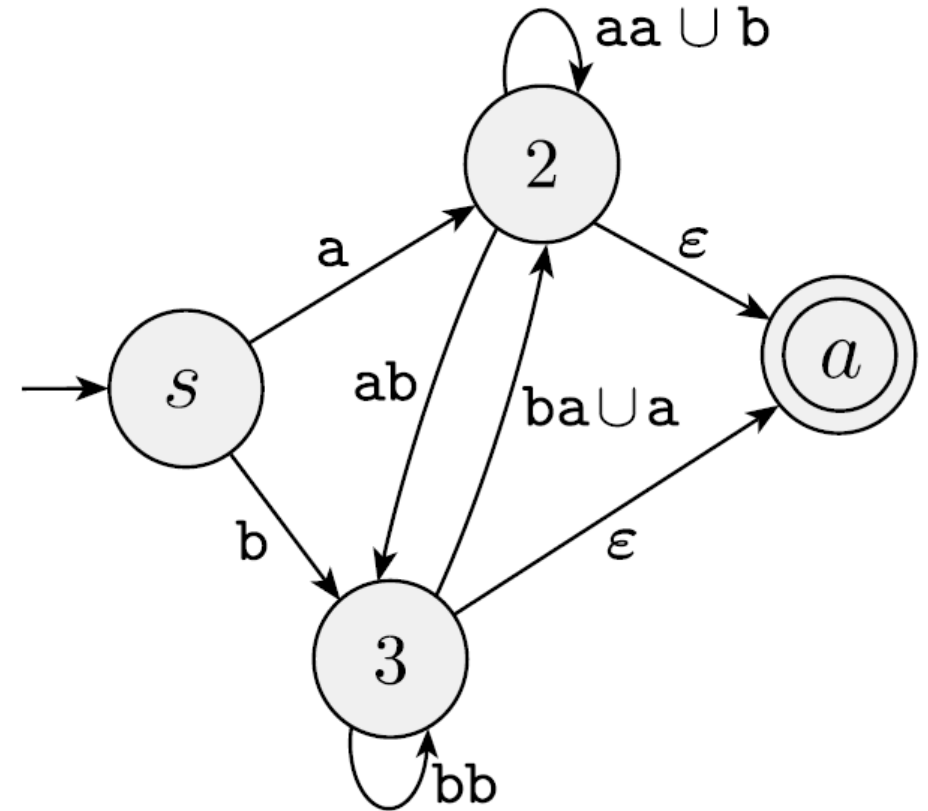
1) $s \rightarrow 2 \rightarrow a$

2) $s \rightarrow 2 \rightarrow 3$

3) $3 \rightarrow 2 \rightarrow a$

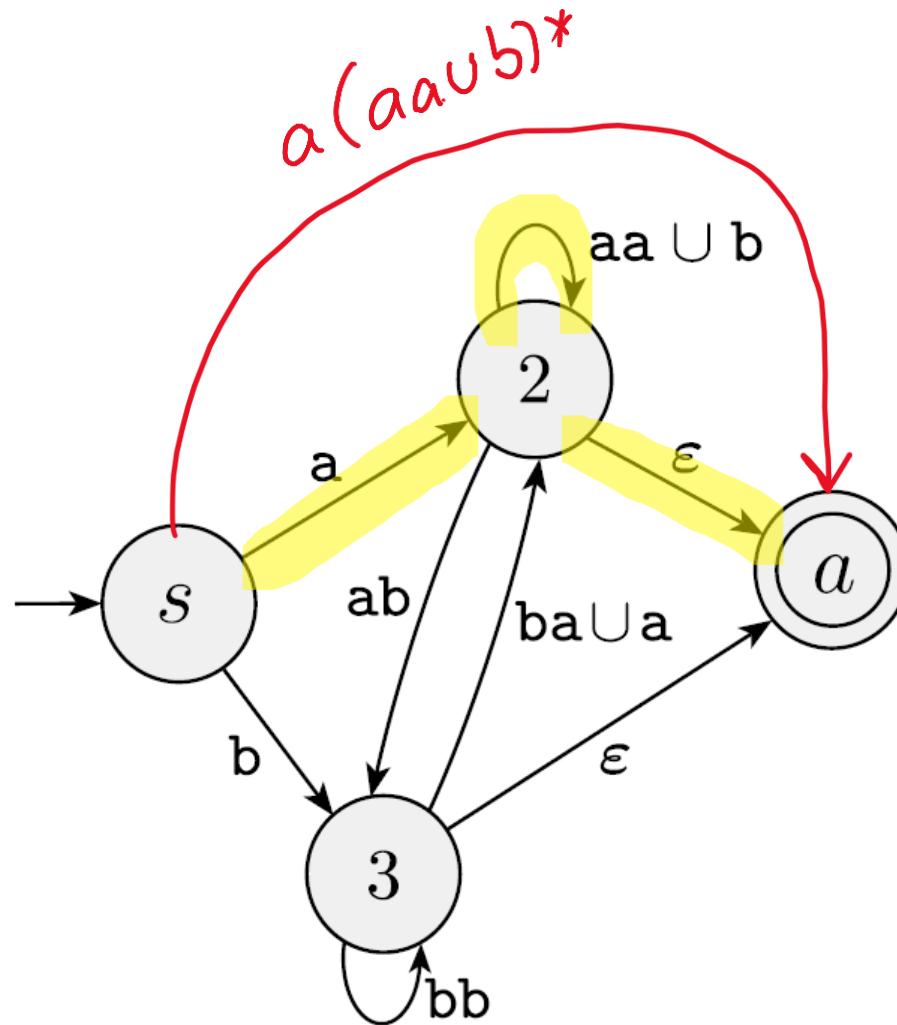
4) $3 \rightarrow 2 \rightarrow 3$

Before removing state 2, we need to add alternative transitions to cover the above paths.



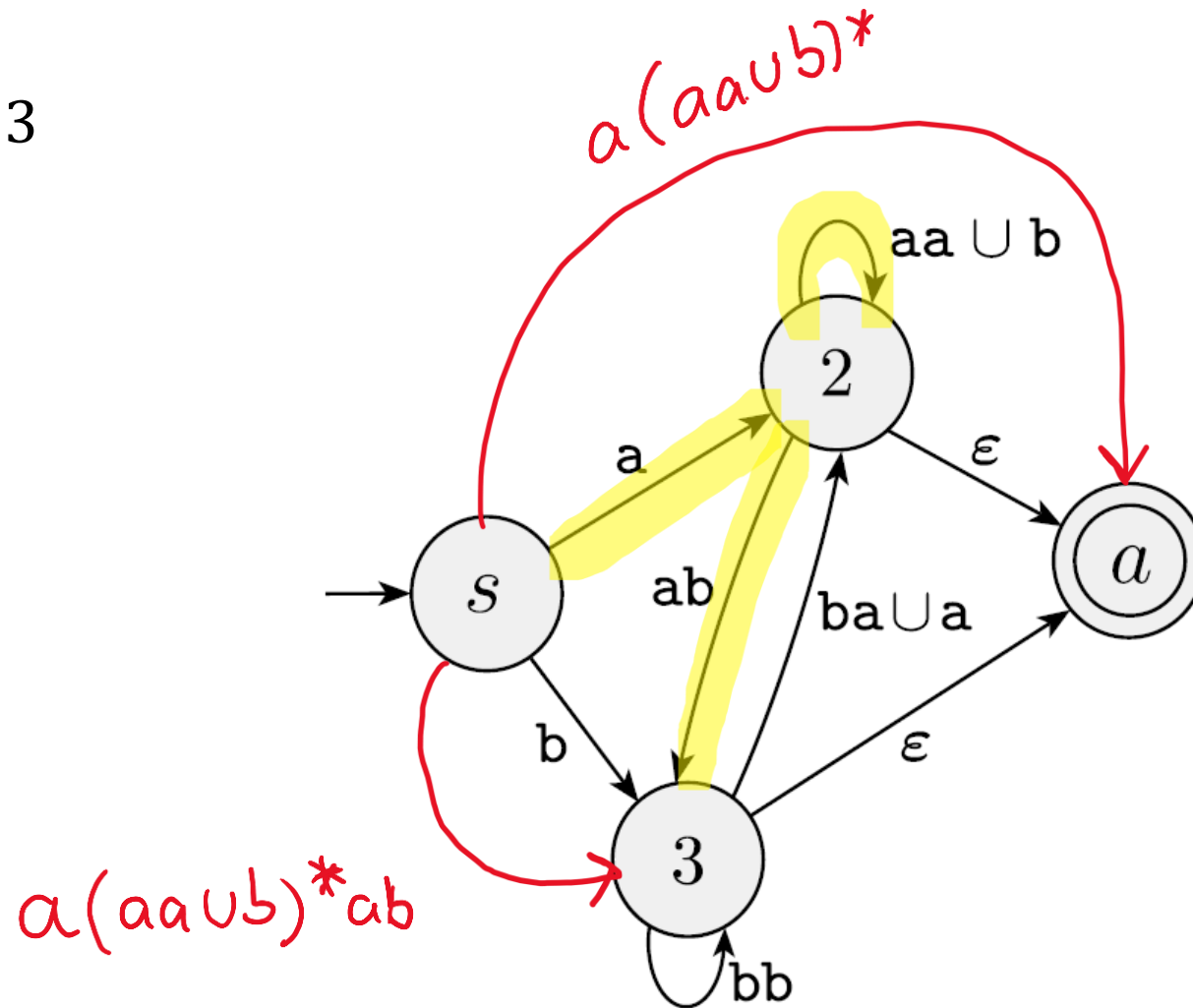
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

$s \rightarrow 2 \rightarrow a$



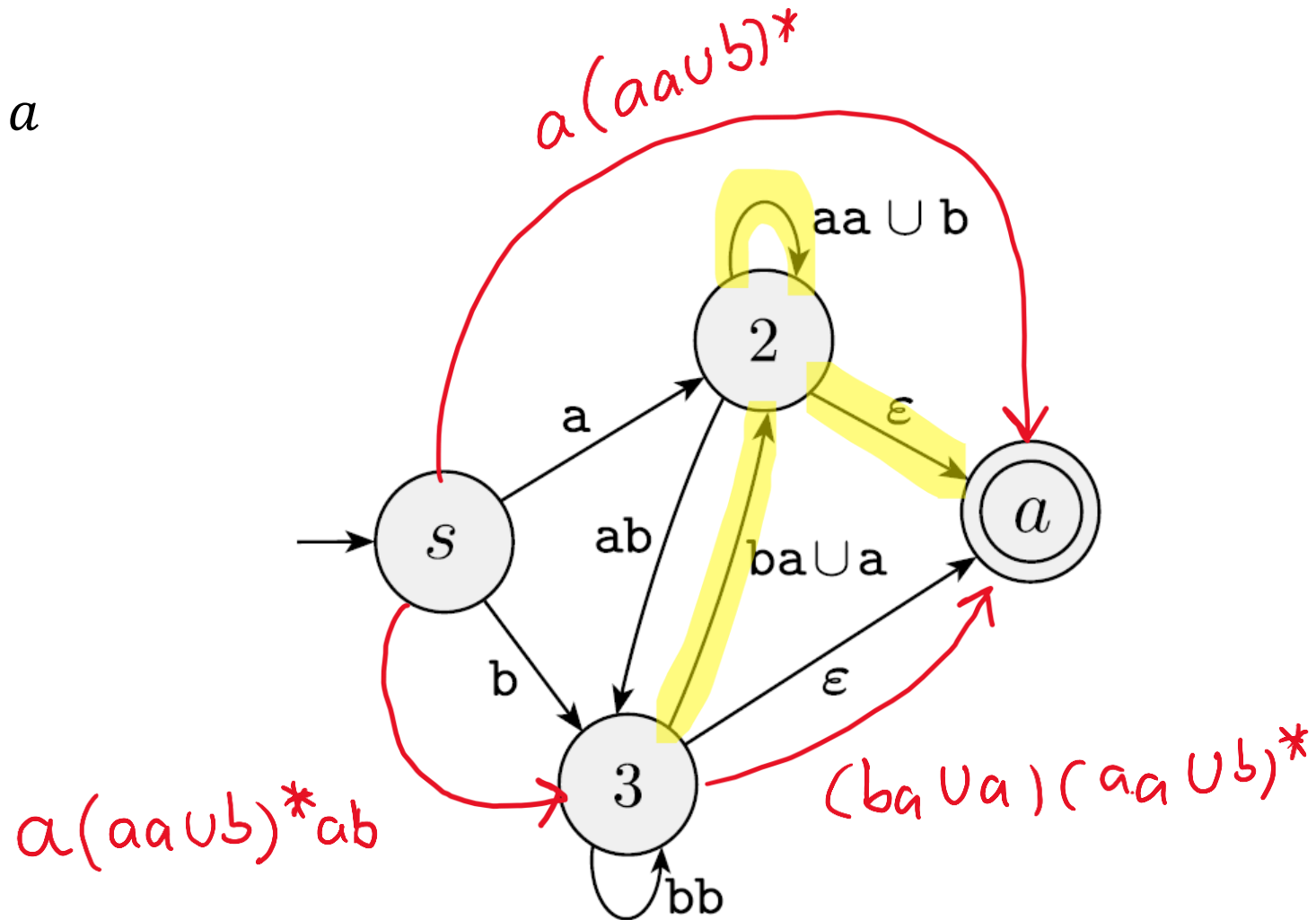
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

$s \rightarrow 2 \rightarrow 3$



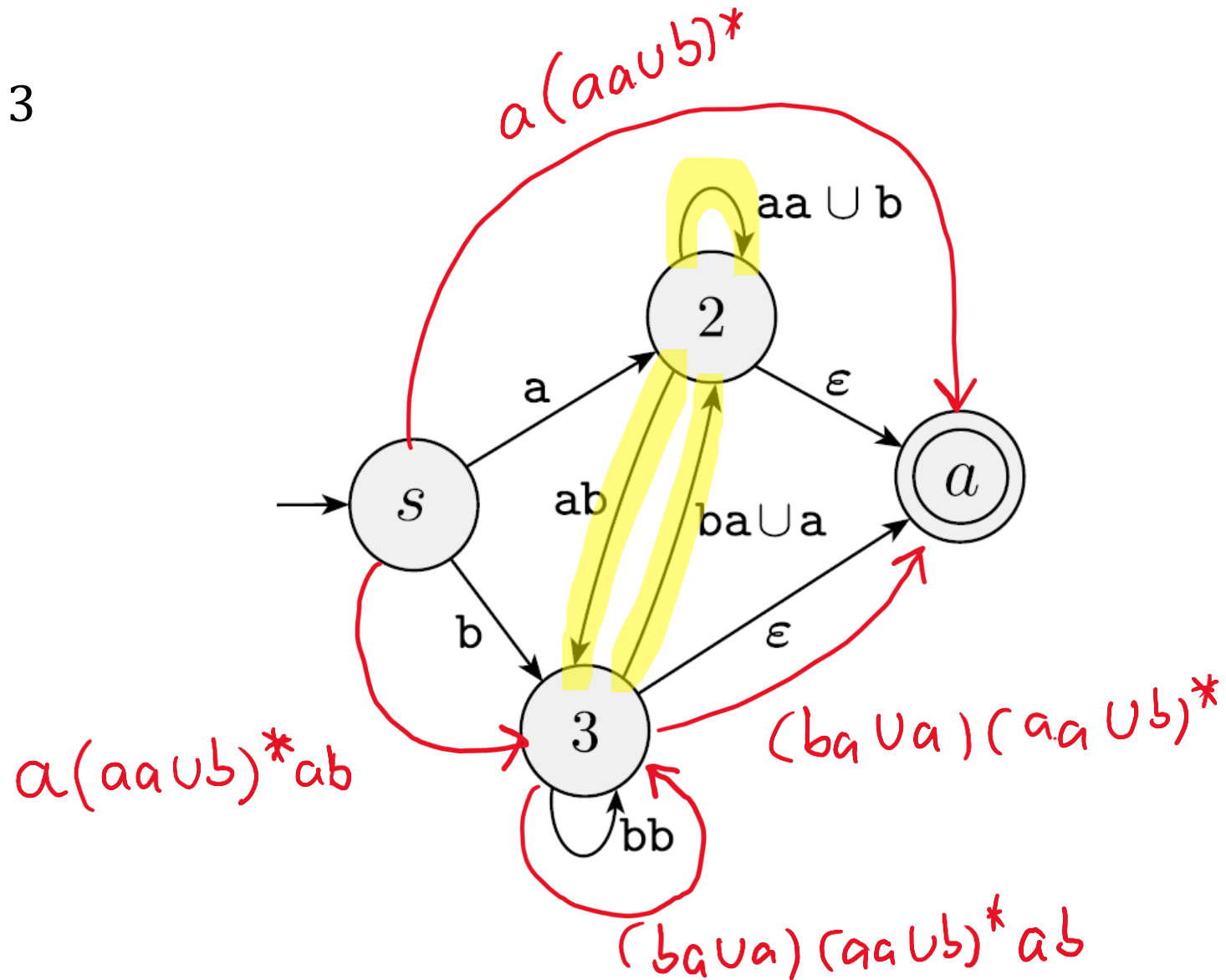
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

$3 \rightarrow 2 \rightarrow a$



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

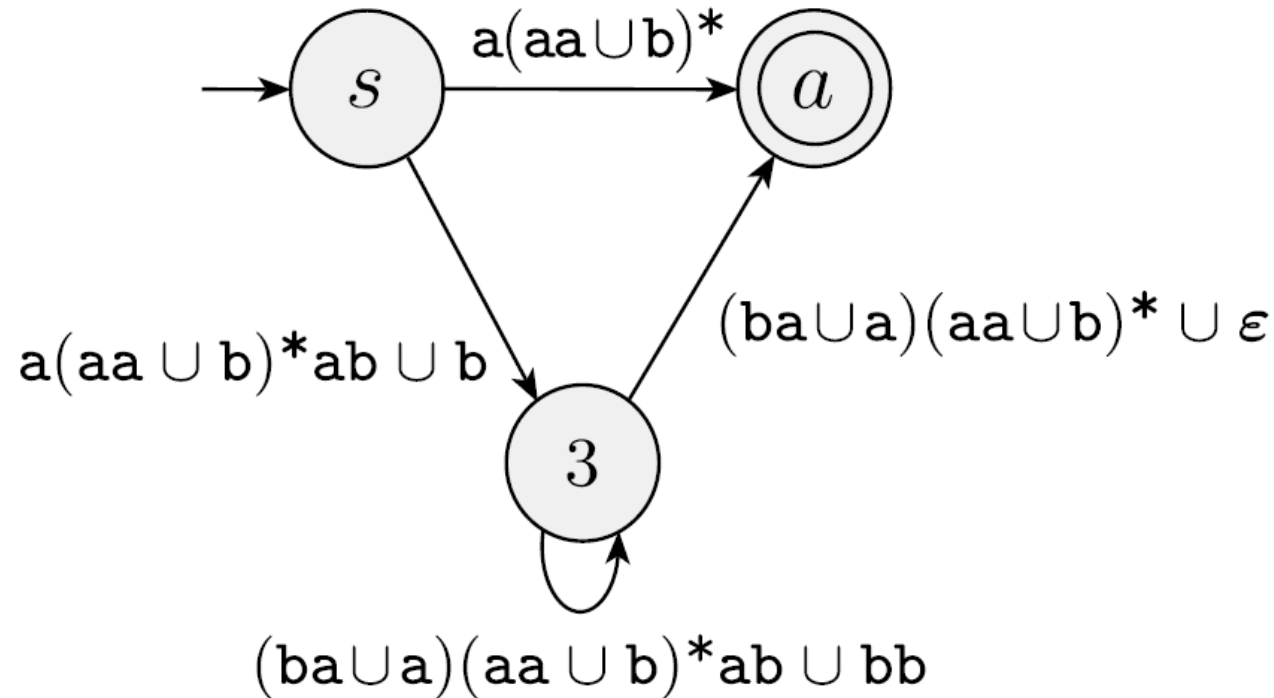
$3 \rightarrow 2 \rightarrow 3$



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

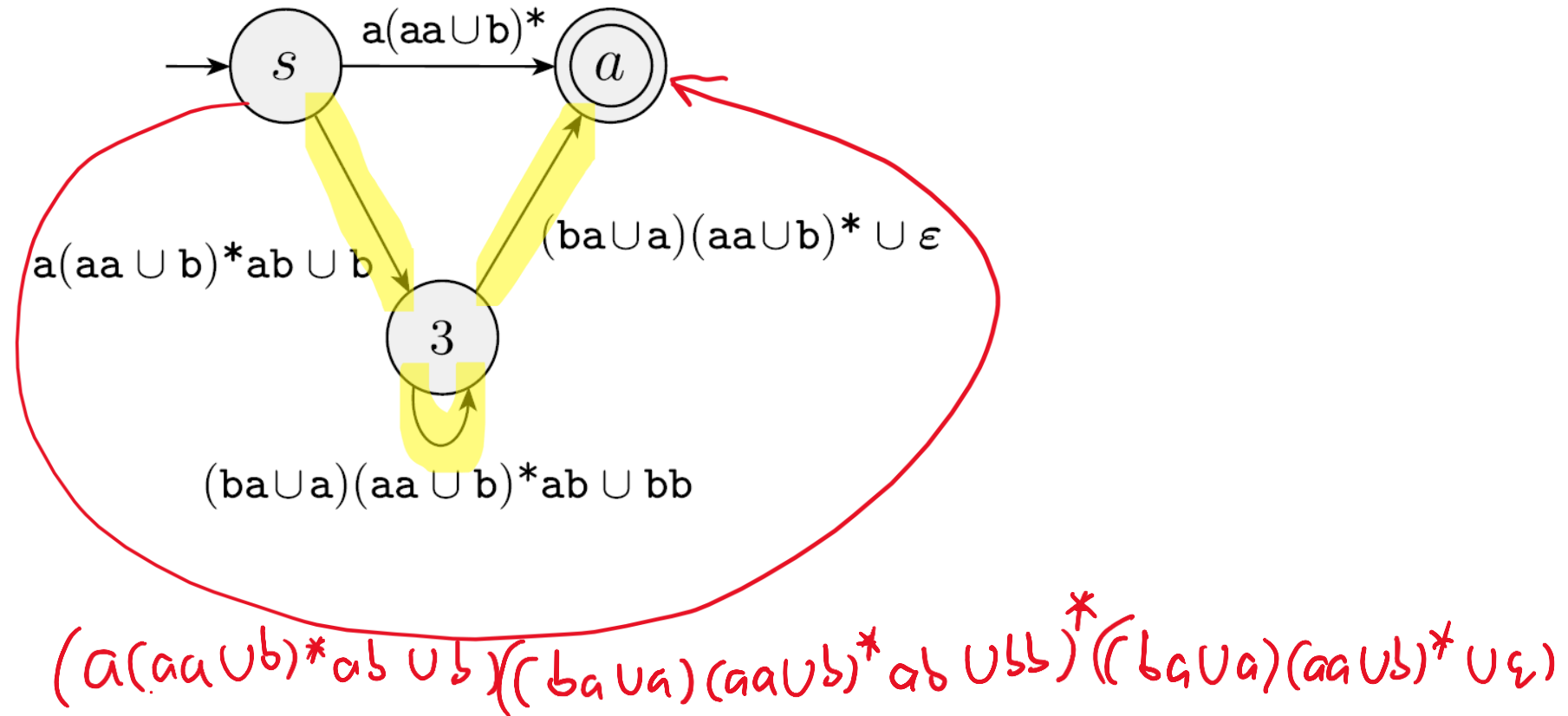
After adding the alternative transitions, we can safely remove state 2.

As before, we combine two transitions with the same origin and the same destination into a single transition by unioning their labels.



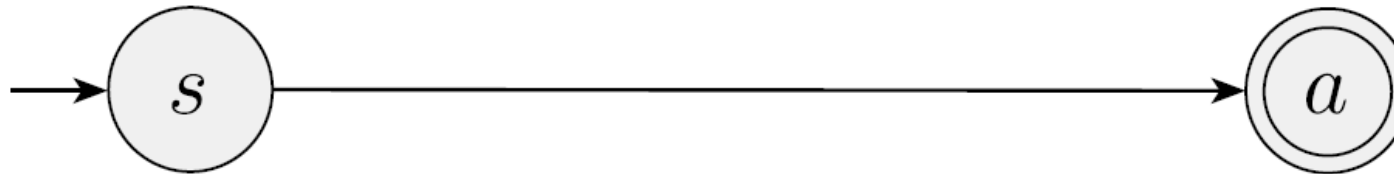
NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

Step 4: Remove State 3.



NFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

We now have an GNFA with only the start state and the accept state and a single transition connecting them. The regular expression labelling the transition is equivalent to the original NFA.



$$(a(aa \cup b)^*ab \cup b)((ba \cup a)(aa \cup b)^*ab \cup bb)^*((ba \cup a)(aa \cup b)^* \cup \epsilon) \cup a(aa \cup b)^*$$

Complementing a regular language

- Even though complementation is not considered a regular operation, it can be shown that the class of regular languages is closed under complementation.
- Given a DFA M , we construct another DFA M' such that $L(M') = \Sigma^* - L(M)$ by setting the final states in M' to be $Q - F$, where F is the set of final states in M .
- Note that, generally, the above method only works with deterministic finite automata. If the given automaton M is nondeterministic, to complement that automaton, we would need to determinize it first (i.e. convert M into an equivalent DFA before applying the above procedure).

Decision Problems on Finite Automata

- **Acceptance problem on DFA/NFA:** Given a DFA/NFA M and string w , determine whether M accepts w or not.
 - This can be seen as a graph traversal problem.
 - Determine whether there is a path in the state diagram of M based on string w to a final state.
- **Emptiness problem on DFA/NFA:** Given a DFA/NFA M , determine whether $L(M) = \emptyset$ or not.
 - This can be seen as a graph reachability problem.
 - Check whether there is a path from the initial state to a final state. Such a path exists if and only if $L(M) \neq \emptyset$.
- **Universality problem on DFA/NFA :** Given a DFA/NFA M , determine whether $L(M) = \Sigma^*$ or not.
 - Construct an automaton M' such that $L(M') = \Sigma^* - L(M)$ and then check whether $L(M) = \emptyset$ or not.

Decision Problems on Finite Automata

- **Subset problem on DFA/NFA:** Given DFAs/NFAs M_1 and M_2 , determine whether $L(M_1) \subseteq L(M_2)$ or not.
 - Since $L(M_1) \subseteq L(M_2)$ iff $(\Sigma' - L(M_2)) \cap L(M_1) = \emptyset$, we can construct an automaton M_3 such that $L(M_3) = (\Sigma' - L(M_2)) \cap L(M_1)$ and then determine whether $L(M_3) = \emptyset$ or not.
- **Equality problem on DFA/NFA:** Given DFAs/NFAs M_1 and M_2 , determine whether $L(M_1) = L(M_2)$ or not.
 - $L(M_1) = L(M_2)$ if and only if $L(M_1) \subseteq L(M_2)$ and $L(M_2) \subseteq L(M_1)$.

Decision Problems on Regular Expressions

- **Acceptance problem on RE:** Given an RE ϕ and string w , determine whether $w \in L(\phi)$.
- **Emptiness problem on RE:** Given an RE ϕ , determine whether $L(\phi) = \emptyset$ or not.
- **Universality problem on RE:** Given an RE ϕ , determine whether $L(\phi) = \Sigma^*$ or not.
- **Subset problem on RE:** Given REs ϕ_1 and ϕ_2 , determine whether $L(\phi_1) \subseteq L(\phi_2)$ or not.
- **Equality problem on RE:** Given REs ϕ_1 and ϕ_2 , determine whether $L(\phi_1) = L(\phi_2)$.

These problems can be solved by converting the given regular expressions to equivalent DFAs/NFAs first.

Non-Regular Languages

How do we show a language is not regular?

- Remember, to show a language *is* regular, we give a DFA.
- To show a language is *not* regular, we must give a proof.
- It is not enough to say that “We couldn’t find a DFA for it. So, it must be non-regular.”

Two examples: Here $\Sigma = \{0,1\}$.

1. Let $B = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$

Is B regular?

2. Let $C = \{w \mid w \text{ has equal numbers of 01 and 10 substrings}\}$

$0101 \notin C$ $0110 \in C$

Is C regular?

Method for Proving Non-regularity

Pumping Lemma: For every regular language A , there is a positive integer p (the “pumping length”) such that

if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

$$1) \ xy^iz \in A \text{ for all } i \geq 0 \qquad y^i = \underbrace{yy \cdots y}_i$$

$$2) \ y \neq \varepsilon$$

$$3) \ |xy| \leq p$$

Informally: A is regular $\rightarrow$ every long string in A can be pumped and the result stays in A .

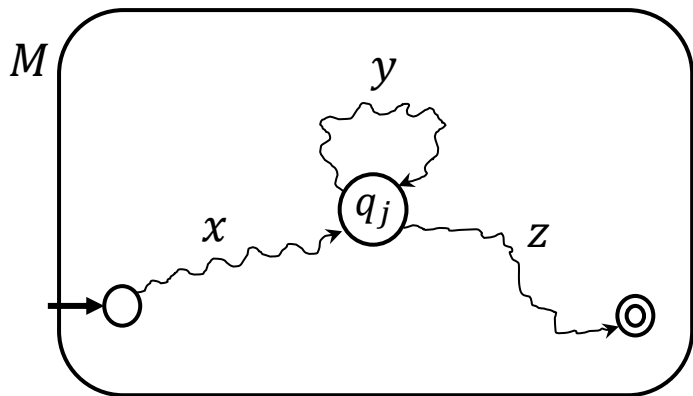
Method for Proving Non-regularity

Pumping Lemma: For every regular language A , there is a positive integer p (the “pumping length”) such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^iz \in A$ for all $i \geq 0$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Proof: Let DFA M recognize A . Let p be the number of states in M .

Pick $s \in A$ where $|s| \geq p$. There must be some state that repeats in the computation path M on s . Let q_j be the first state that repeats in the path.



$$s = \frac{x}{q_j} \mid \frac{y}{q_j} \mid z$$

M will repeat a state q_j when reading s because s is so long.

$$\frac{x}{q_j} \mid \frac{y}{q_j} \mid \frac{y}{q_j} \mid z \text{ is also accepted}$$

Example 1

Let $B = \{0^k 1^k \mid k \geq 0\}$

Show: B is not regular

Proof by Contradiction:

Assume (to get a contradiction) that B is regular.

The pumping lemma gives p as above. Let $s = 0^p 1^p \in B$.

Pumping lemma says that we can divide $s = xyz$ satisfying the 3 conditions.

This means that $y = 0^k$ for some $k \geq 1$.

$$s = \frac{000 \dots 000111 \dots 111}{\begin{array}{ccc} x & y & z \\ \leftarrow \leq p \rightarrow \end{array}}$$

But $xyyz$ has excess 0s and thus $xyyz \notin B$ contradicting the pumping lemma.

Therefore, our assumption (B is regular) is false.

We conclude that B is **not** regular.

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^i z \in A$ for all $i \geq 0$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Example 2

Let $C = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$

Show: C is not regular

Proof by Contradiction:

Assume (to get a contradiction) that C is regular.

The pumping lemma gives p as above. Let $s = 0^p 1^p \in C$.

Pumping lemma says that we can divide $s = xyz$ satisfying the 3 conditions.

This means that $y = 0^k$ for some $k \geq 1$.

$$s = \begin{array}{c} 000 \dots 000111 \dots 111 \\ \hline \begin{array}{ccc} x & y & z \\ \leftarrow \leq p \rightarrow \end{array} \end{array}$$

But $xyyz$ has excess 0s and thus $xyyz \notin C$ contradicting the pumping lemma.

Therefore, we conclude that C is **not** regular.

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^i z \in A$ for all $i \geq 0$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Example 2 (Alternative proof)

Technique: Combine closure properties with the Pumping Lemma.

Let $C = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$

Show: C is not regular

Proof by Contradiction:

Assume (for contradiction) that C is regular.

We know that 0^*1^* is regular so $C \cap 0^*1^*$ is regular (closure under intersection).

But $B = C \cap 0^*1^*$ and we already showed B is not regular. Contradiction!

Therefore, our assumption is false, so C is not regular.

Example 3

Let $F = \{ww \mid w \in \Sigma^*\}$.

Show: F is not regular

Proof by Contradiction:

Assume (for contradiction) that F is regular.

The pumping lemma gives p as above. Need to choose $s \in F$. Which s ?

Try $s = 0^p 0^p \in F$. But that s can be pumped and stay inside F . Bad choice!

$$s = \frac{000 \dots 000000 \dots 000}{\begin{array}{ccc} x & y & z \\ \leftarrow \leq p \rightarrow \end{array}} \quad y = 00$$

Try $s = 0^p 1 0^p 1 \in F$. Hence, $y = 0^k$ for some $k \geq 1$. But $xyyz \notin F$. Contradiction!

$$s = \frac{000 \dots 001000 \dots 001}{\begin{array}{ccc} x & y & z \\ \leftarrow \leq p \rightarrow \end{array}}$$

Therefore, F is not regular.

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^i z \in A$ for all $i \geq 0$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Example 4

Let $D = \{1^{n^2} \mid n \geq 0\}$.

Show: D is not regular

Proof by Contradiction:

Assume (for contradiction) that D is regular.

Let p be the pumping length. Let s be the string 1^{p^2} .

By the pumping lemma, $s = 1^{p^2}$ can be split into xyz , where $|xy| \leq p$ and $|y| \geq 1$, such that $xy^iz \in D$ for all $i \geq 0$.

In particular, $xyyz \in D$. Consider the length $xyyz$.

$$|xyyz| = |xyz| + |y| = p^2 + |y|$$

Since $1 \leq |y| \leq p$, we have

$$p^2 < |xyyz| \leq p^2 + p < (p+1)^2$$

This means that the length of $xyyz$ is not a square number. Hence, $xyyz \notin D$.

Hence, our assumption that D is regular is false. Therefore, D is not regular.

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^iz \in A$ for all $i \geq 0$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Example 5 (pumping down)

Let $F = \{0^j 1^k \mid j > k \geq 0\}$

Show: F is not regular

Proof by Contradiction:

Assume (to get a contradiction) that F is regular.

Let p be the pumping length. Let $s = 0^{p+1}1^p \in F$.

By the pumping lemma, $s = 0^{p+1}1^p$ can be split into xyz , where $|xy| \leq p$ and $|y| \geq 1$, such that $xy^i z \in F$ for all $i \geq 0$.

This means that $y = 0^k$ for some $k \geq 1$.

$$s = \frac{000 \cdots 000111 \cdots 111}{\begin{array}{c} x \quad y \quad z \\ \leftarrow \leq p \rightarrow \end{array}}$$

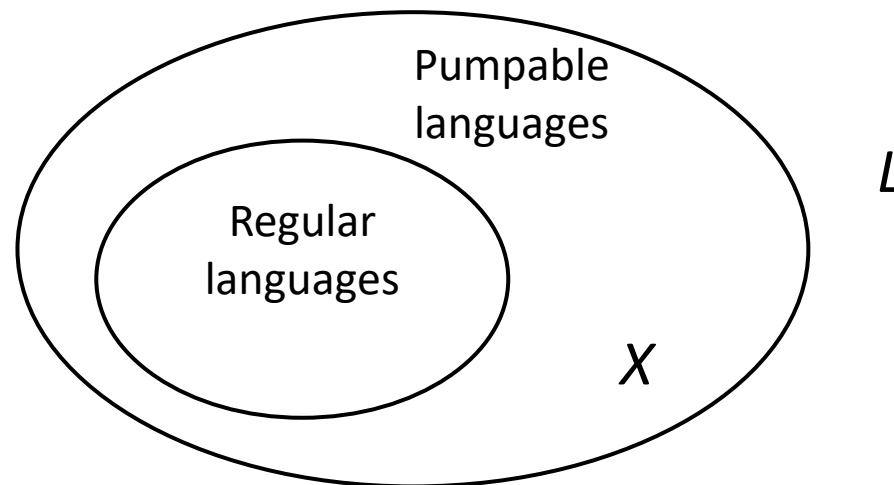
In particular, $xy^0 z = xz$ is in F . But this cannot be the case, because the number of zero in xz is not greater than the number of one. Therefore, our assumption (F is regular) is false. We conclude that F is **not** regular.

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^i z \in A$ for all $i \geq 0$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Pumpable non-regular languages

- Basically, the pumping lemma asserts that every regular language can be “pumped” (i.e. satisfying the condition in the pumping lemma).
- We use this lemma to show that if a language L cannot be pumped, then it is not regular.
- Be aware that the converse is **not** generally true. There are “pumpable” languages that are not regular.



Pumpable non-regular languages

$m \neq 1$ iff $m = 0$ or $m \geq 2$

- Consider the language $X = X_1 \cup X_2$, which is the union of the following disjoint sets:

$$X_1 = \{ab^n c^n \mid n \geq 0\} \quad \text{and} \quad X_2 = \{a^m w \mid \underline{m \neq 1} \text{ and } w \in \{b, c\}^*\}$$

- X is clearly not regular. But we can show that X can be pumped with pumping length $p = 2$.
- Let s be any string in X of length p or more. We can show that s can be split into xyz such that $|y| \geq 1$, $|xy| \leq p$, and $xy^i z \in X$ for all $i \geq 0$.
- We set $x = \varepsilon$ and pick y and z based on s as follows. Here, $n, k \geq 0$ and $w \in \{b, c\}^*$.

$$s = \underbrace{a}_{y} \underbrace{b^n c^n}_z$$

$$s = \underbrace{aa}_{y} \underbrace{w}_z$$

$$s = \underbrace{aaaa}_{y} \underbrace{a^k w}_z$$

$$s = \underbrace{b}_{y} \underbrace{w}_z$$

$$s = \underbrace{c}_{y} \underbrace{w}_z$$

- We can check in each case that $xy^i z$ is in X_1 or X_2 , for all $i \geq 0$.
Thus, this non-regular language satisfies the condition in the pumping lemma.

Myhill-Nerode Theorem

Recall the definition of an equivalence relation.

- **Definition (Equivalence Relations).** Suppose S is a set. A (binary) relation R on S is said to be an equivalence relation if and only if
 - R is reflexive: $x R x$, for all $x \in S$;
 - R is symmetric: $x R y$ implies $y R x$, for all $x, y \in S$;
 - R is transitive: $x R y$ and $y R z$ implies $x R z$, for all $x, y, z \in S$.
- **Example.** Suppose $S = \{0,1,2,3,4,5,6,7,8\}$ and $\sim$ is defined by
$$x \sim y \text{ if and only if } x \bmod 3 = y \bmod 3, \quad \text{for all } x, y \in S.$$
 - Thus, for example, $1 \sim 4$ and $1 \sim 7$ but $1 \not\sim 2$ and $1 \not\sim 0$.
 - It is easy to check that $\sim$ is an equivalence relation.

Myhill-Nerode Theorem

- Based on a given equivalence relation R , if we group elements in the domain that are related in R together, we can see that the relation R partitions the domain into a number of subsets of elements, called *equivalence classes*.
- **Definition (Equivalence Classes).** Given an equivalence relation R on S , for each element $x \in S$, the equivalence class in R containing x , denoted by $[x]^R$, is defined by
$$[x]^R = \{y \in S \mid xRy\}$$
- When it is clear which relation we are referring to, we shall omit the superscript R from $[x]^R$ and simply write $[x]$.

Myhill-Nerode Theorem

- The following proposition is easily show from the properties of equivalence relations.
- **Proposition.** Suppose R is an equivalence relation on S . For all $x, y \in S$,
 - $x \in [x]$,
 - $x \in [y]$ if and only if $y \in [x]$,
 - xRy if and only if $[x] = [y]$.
- This can be easily shown from the properties of equivalence relations.
- Suppose $\sim$ is the relation from the previous example. It has 3 distinct equivalence classes, namely

$$[0] = [3] = [6] = \{0, 3, 6\}$$

$$[1] = [4] = [7] = \{1, 4, 7\}$$

$$[2] = [5] = [8] = \{2, 5, 8\}$$

Myhill-Nerode Theorem

- **Definition (Finite Index).** An equivalence relation R is said to be of **finite index** if and only if there are finitely many equivalence classes in R .
- Given an equivalence relation R of finite index, the **index of R** , sometimes denoted by $\#R$, is the number of distinct equivalence classes of R .
- Thus, the relation $\sim$ defined in the previous example is of finite index, as it has 3 distinct equivalence classes.

Myhill-Nerode Theorem

- **Definition (Myhill-Nerode Relations).** Given a language L , a **Myhill-Nerode relation** for L is any (binary) relation $\equiv$ on Σ^* satisfying all of the following
 - $\equiv$ is an **equivalence relation**.
 - $\equiv$ is a **right congruence** with respect to concatenation, i.e. for any strings $x, y \in \Sigma^*$ and character $a \in \Sigma$,
$$\text{if } x \equiv y, \text{ then } (x \circ a) \equiv (y \circ a).$$
 - $\equiv$ **refines** L , i.e. for any string $x, y \in \Sigma^*$,
$$\text{if } x \equiv y, \text{ then } x \in L \leftrightarrow y \in L.$$
 - $\equiv$ is of **finite index**.

Myhill-Nerode Theorem

- **Example.** Show that there is Myhill-Nerode relation for the language

$$L = \{x \in \Sigma^* \mid x \text{ contains an odd number of } 1\}.$$

- Let us say that two integers have the **same parity** if and only if they are either both odd or both even.
- Define the relation $\equiv$ on Σ^* : $x \equiv y$ iff the number of 1 in x and the number of 1 in y have the same parity. For instance, $110 \equiv 010111$ but $110 \not\equiv 01011$.
- It is easy to check that
 - $\equiv$ is an equivalence relation;
 - $\equiv$ is a right congruence, i.e. $x \equiv y$ implies $xa \equiv ya$;
 - $\equiv$ refines L , i.e. $x \equiv y$ implies $x \in L \leftrightarrow y \in L$;
 - $\equiv$ is of finite index (in fact, $\equiv$ has 2 equivalence classes)

Myhill-Nerode Theorem

- A given language L might or might not have a Myhill-Nerode relation. One famous result is that a Myhill-Nerode relation for L exists if and only if L is a regular language.
- **Myhill-Nerode Theorem.** Suppose L is a language. L is regular if and only if there exists a Myhill-Nerode relation for L .
- **(Proof idea)** If L is regular, there is a DFA M recognizing L . We can define a Myhill-Nerode relation for L from the DFA M .
Conversely, if L has a Myhill-Nerode relation $\equiv$, then we can construct a DFA M recognizing L where each equivalence class of $\equiv$ becomes a state in M .
- Myhill-Nerode Theorem can be used to prove that a language is not regular, as we shall show in the proposition below. First, let us prove a useful property of a right congruence.

Myhill-Nerode Theorem

- **Lemma.** Suppose $\equiv$ is an equivalence relation on Σ^* which is a **right congruence**, i.e. for all strings $x, y \in \Sigma^*$ and character $a \in \Sigma$,

$$x \equiv y \text{ implies } (x \circ a) \equiv (y \circ a).$$

Then, for any strings $x, y, z \in \Sigma^*$,

$$x \equiv y \text{ implies } (x \circ z) \equiv (y \circ z).$$

- This lemma can be proved by induction on the length of string z (see handout for the proof).

Myhill-Nerode Theorem

- To apply Myhill-Nerode Theorem to show that a language L is *not* regular, we can proceed as follows:

Step 1. Assume that L is regular and hence has a Myhill-Nerode relation, say $\equiv$.

Step 2. Construct infinitely many (distinct) strings $x_1, x_2, x_3, \dots$ such that $x_i \not\equiv x_j$ for all $i \neq j$. To show that $x_i \not\equiv x_j$, we only need to find just one string z such that $x_i \circ z \in L$ but $x_j \circ z \notin L$, or vice versa.

Step 3. Conclude from Step 2 that the strings $x_1, x_2, x_3, \dots$ are all in different equivalence classes, which would mean that there are infinitely many equivalence classes. This contradicts the fact that $\equiv$ is a Myhill-Nerode relation, which must be of finite index. Therefore, L is *not* regular.

Myhill-Nerode Theorem

- **Example.** Show that the language $L = \{0^k 1^k \mid k \geq 0\}$ is *not* regular.
- Assume that L is regular. Hence, by Myhill-Nerode Theorem, there must be a Myhill-Nerode relation, say $\equiv$, for L .
- Consider the strings
 $\epsilon, \quad 0, \quad 00, \quad 000, \quad 0000, \quad 00000, \quad \dots, \quad 0^n, \quad \dots$
- Consider any two strings 0^i and 0^j where $i \neq j$. If we append these two strings by 1^i , we have $0^i 1^i \in L$ but $0^j 1^i \notin L$. This means $0^i \not\equiv 0^j$.
- Hence, the strings above are all in different classes, contradicting the fact that $\equiv$ is a Myhill-Nerode relation, which must be of finite index.
- Therefore, L is *not* regular.

Myhill-Nerode Theorem

- **Example.** Show that the language $L = \{w \in \Sigma^* \mid w \text{ is a palindrome}\}$ is *not* regular.
- Assume that L is regular. Hence, by Myhill-Nerode Theorem, there must be a Myhill-Nerode relation, say $\equiv$, for L .
- Consider the strings
1, 01, 001, 0001, 00001, ... , $0^n 1$, ...
- Consider any two strings $0^i 1$ and $0^j 1$ where $i \neq j$. If we append these two strings by 0^i , we have $0^i 1 0^i \in L$ but $0^j 1 0^i \notin L$. This means $0^i 1 \not\equiv 0^j 1$.
- Hence, the strings above are all in different classes, contradicting the fact that $\equiv$ is a Myhill-Nerode relation, which must be of finite index.
- Therefore, L is *not* regular.

Myhill-Nerode Theorem

- **Example.** Show that the language $L = \{1^{k^2} \mid k \geq 0\}$ is *not* regular.
- Assume that L is regular. Hence, by Myhill-Nerode Theorem, there must be a Myhill-Nerode relation, say $\equiv$, for L .
- Consider the strings

$$1^{0^2}, 1^{1^2}, 1^{2^2}, 1^{3^2}, \dots, 1^{n^2}, \dots$$

- Consider any two strings 1^{i^2} and 1^{j^2} where $0 \leq i < j$. If we append these two strings by 1^{2i+1} , we have

$$\begin{aligned} 1^{i^2} 1^{2i+1} &= 1^{i^2+2i+1} = 1^{(i+1)^2} \\ 1^{j^2} 1^{2i+1} &= 1^{j^2+2i+1} \end{aligned}$$

The first string $1^{(i+1)^2}$ is in L . We will show that the second string 1^{j^2+2i+1} is *not* in L .

Myhill-Nerode Theorem

- Since $0 < 2i + 1$ and $i < j$, we have

$$\begin{aligned} j^2 &< j^2 + 2i + 1 < j^2 + 2j + 1 \\ j^2 &< j^2 + 2i + 1 < (j + 1)^2. \end{aligned}$$

Since $j^2 + 2i + 1$ is strictly between j^2 and $(j + 1)^2$, it follows that $j^2 + 2i + 1 \neq k^2$ for all integers k . This implies 1^{j^2+2i+1} is not in L . Hence, $1^{i^2} \not\equiv 1^{j^2}$.

- Hence, the strings above are all in different classes, contradicting the fact that $\equiv$ is a Myhill-Nerode relation, which must be of finite index.
- Therefore, L is *not* regular.